Robustness, Typed Hypernets, and Plugging

A mathematical commentary on the Lean development

Companion to `danghica/hypernets-lean`

23 October 2026

Source snapshot: `ce31471`; Lean `4.35.0`

Abstract

This document explains every definition and proved statement in the current Lean development, including its regression examples. It gives proofs in mathematical English and links each declaration to its complete source fragment at a fixed Git revision. The abstract counting-simulation soundness theorem is proved outright. The forward and converse robustness arguments are proved relative to an explicit specimen interface. The concrete development constructs finite typed graphs, proves well-formed shallow plugging, path and isomorphism results, operation-path uniqueness, finite boxed hierarchies, and addressed deep replacement. The connection to a concrete universal abstract machine and the verification of the specimen interface remain outstanding; the development is therefore not yet a formalization of the paper's full Theorem 6.

Contents

1 Scope, notation, and the status of the formalization **4**
1.1 Conventions 4
1.2 Dependency order 5

2 Machines and finite executions **5**

3 Cost relations and state refinement **8**

4 Counting simulation and its soundness **10**

5 Typed observations and contextual lifting **12**

6 The abstract specimen interface **13**

7 The forward robustness argument **16**

8 Opposite specimens and the converse argument **17**

9 State and contextual consequences **19**

10 Semantic examples and regression proofs **21**
10.1 A countdown machine 21
10.2 A crash and an infinite loop 22
10.3 The zero-step converse case 23

11 Finite carriers and concrete interfaced graphs **25**
11.1 Enumerations and bijections 25
11.2 Ports, signatures, and graph invariants 26
11.3 Elementary graph constructors 28

12 Shallow plugging, including self-loop holes **29**
12.1 Carriers and boundary identification 29
12.2 Preservation of incidence invariants 30
12.3 Labels, typing, and the certified result 31

13 Paths, renaming, and isomorphism **32**
13.1 Paths and their behaviour under plugging 32
13.2 Transport along carrier bijections 34
13.3 Graph isomorphism and shallow context congruence 35

14 Concrete uniqueness of operation paths **36**

15 Typed boxes, deep occurrences, and contexts **38**

15.1 Relabelling without changing incidence . . . 38
15.2 The linear wire and label discipline . . . 38
15.3 Box construction and promotion of the depth bound . . . 40
15.4 Finite occurrence addresses at arbitrary depth . . . 40
15.5 Hole lookup and the context condition . . . 41

16 Addressed deep replacement and concrete locality 42
16.1 Changing one box body . . . 42
16.2 Typed sites and replacement at a site . . . 42

17 Concrete graph examples and regression proofs 43
17.1 Self-loop gluing and the necessary reflection restriction . . . 43
17.2 Values, boxes, and distinguished occurrence addresses . . . 44
17.3 An input-rooted operation path . . . 45
17.4 Filling a hole inside a certified context . . . 46

18 Proof checking and remaining obligations 47
18.1 The two axiom audits . . . 47
18.2 Reproducing the Lean and document checks . . . 47
18.3 What remains for the paper's concrete theorem . . . 48

1 Scope, notation, and the status of the formalization

The abstract part of the development follows the proof in Appendix F of [1]. Its completed mathematical core is the soundness of counting simulation, corresponding to Proposition 37. The remaining argument is expressed through a structure of abstract specimens. Given that structure and its laws, Lean proves both directions of the simulation construction and their contextual consequences. Constructing this structure from the paper's actual hypernets is still a separate task. The concrete graph part now supplies finite interfaced graphs, certified shallow plugging, renaming and path invariance, a typed boxed hierarchy, and addressed deep replacement. These constructions and their regression examples are explained after the abstract development.

There are three distinct kinds of assertion below. A *definition* specifies a mathematical object or predicate. A *proved statement* is established by a Lean proof term. An *interface law* is a hypothesis carried by a structure; the current code uses that hypothesis but does not prove it for concrete hypernets. In particular, the laws of a specimen system must not be read as established theorems about the universal abstract machine.

All code links point to revision `ce31471`, rather than to the moving `main` branch. Each link highlights the declaration together with its proof or defining fields. Namespace prefixes in link labels disambiguate declarations with the same short name. Anonymous Lean `example` declarations are explained alongside the named theorems.

1.1 Conventions

Fix a set of states X and a machine M on X. Write $s \to t$ for its one-step transition relation, $s \longrightarrow^n t$ for exactly n transitions, and $\mathsf{Final}(s)$ for successful finality. Natural numbers include zero. A cost preorder Q is written infix: $n\,Q\,m$. The source count is always the left argument and the target count the right argument. Write $s \preceq_Q t$ for state refinement, and $s \simeq_Q t$ for mutual state refinement.

The paper's three cost parameters (Q, Q', Q'') are named (Q, P, S) in the code and in this commentary. Thus P is the auxiliary relation on the *left* of a simulation candidate, and S is the one on its *right*. We spell out witnesses rather than relying on a convention for composition of relations.

No result assumes that all executions terminate. An arbitrary state can be final, stuck, or capable of further computation. Successful termination is the only qualitative observation in the present state-refinement definition. Neither equality of returned values nor equality of final states across two different source states is required.

1.2 Dependency order

The files are organized as follows. Finite-execution facts in `Transition.lean` support the quantitative relations in `Quantitative.lean` and the soundness argument in `Simulation.lean`. The observation interface in `Contextual.lean` converts state results into contextual ones. The specimen argument in `LocalReasoning.lean` constructs the simulation to which soundness applies. Finally, `Examples.lean` checks substantive edge cases, and `Audit.lean` reports theorem dependencies. The root file `Hypernets.lean` imports the development and its examples; it adds no mathematical declaration.

The concrete branch begins with `Graph/Finite.lean` and `Graph/Basic.lean`. Plugging, paths, and isomorphism support the hierarchy and deep replacement modules. Operation-path uniqueness imports the counted-execution results, providing a concrete use of the abstract machine lemmas. Both branches are imported by the root file.

The exposition follows these dependency orders. Section numbers and theorem numbers here are local to this document; references to the paper are explicitly identified as such.

2 Machines and finite executions

Definition 2.1 (Machine)**.** A machine on X consists of a relation $\to \subseteq X \times X$, a predicate $\mathsf{Final} \subseteq X$, and the law

$$\mathsf{Final}(s) \implies \neg(s \to t) \qquad (s, t \in X).$$

Thus every successful final state has no outgoing transition. No converse is assumed: an unsuccessful normal form is permitted.

Lean: `Machine`

The fields `step` and `final` supply the relation and predicate. The field `final_normal` supplies the displayed law. A machine may have any state type; finiteness or a graphical representation is not required.

Definition 2.2 (Determinism, normality, and stuckness)**.** The machine is *deterministic* if $s \to t_1$ and $s \to t_2$ imply $t_1 = t_2$. A state s is *normal* if $s \to t$ is false for every t. It is *stuck* if it is normal and is not final:

$$\mathsf{Normal}(s) \iff \forall t\, \neg(s \to t), \qquad \mathsf{Stuck}(s) \iff \mathsf{Normal}(s) \wedge \neg\mathsf{Final}(s).$$

Lean: `Machine.Deterministic`

Lean: `Machine.Normal`

Lean: `Machine.Stuck`

Determinism is a separate hypothesis, not a field of every machine. Stuckness distinguishes failure from successful finality even though both are normal forms.

Definition 2.3 (Counted execution)**.** The relation $s \longrightarrow^n t$ is generated inductively by

$$\frac{}{s \longrightarrow^0 s}, \qquad \frac{s \to u \quad u \longrightarrow^n t}{s \longrightarrow^{n+1} t}.$$

Lean: `Machine.Steps`

These are precisely the constructors `zero` and `cons`. The count includes every transition of the machine. There is no separate cost weight for searching or rewriting. In particular, zero transitions cannot change a state.

Lemma 2.4 (Zero-step equality)**.** *If $s \longrightarrow^0 t$, then $s = t$.*

Lean: `Machine.Steps.zero_eq`

Proof. Only the zero constructor can produce an execution with count zero. Its two endpoints are the same state. The step constructor has a positive count and cannot occur. □

Lemma 2.5 (Concatenation)**.** *If $s \longrightarrow^m t$ and $t \longrightarrow^n v$, then $s \longrightarrow^{m+n} v$.*

Lean: `Machine.Steps.append`

Proof. Induct on the first execution. For a zero-step execution, $s = t$, so the second execution is the required one. Otherwise its first transition is $s \to u$ and its remaining execution has some length k. By induction, that remainder followed by the second execution has length $k + n$. Prepending $s \to u$ gives length $(k + n) + 1 = (k + 1) + n$. No determinism is used. □

Lemma 2.6 (An execution from a normal state)**.** *If $s \longrightarrow^n t$ and $\mathsf{Normal}(s)$, then $n = 0$ and $s = t$.*

Lean: `Machine.Steps.normal_eq`

Proof. The zero-step case has both conclusions by definition. A positive execution would begin with a transition out of s, contradicting normality. □

For the following three lemmas, assume determinism.

Lemma 2.7 (Every execution is a prefix of a final execution)**.** *Suppose $s \longrightarrow^m t$, $s \longrightarrow^n z$, and $\mathsf{Final}(z)$. Then there is a natural number r such that*

$$n = m + r \quad \text{and} \quad t \longrightarrow^r z.$$

In particular $m \leq n$.

Lean: `Machine.Steps.prefix_of_final`

Proof. Induct on the execution of length m, allowing n to vary. If $m = 0$, then $t = s$; take $r = n$ and use the given final execution.

Otherwise the first execution starts with $s \to u$. The final execution cannot have length zero: that would make $s = z$ final while an outgoing transition exists. It therefore starts with $s \to u'$. Determinism gives $u = u'$. Apply the induction hypothesis to the two remaining executions. If their lengths are $m - 1$ and $n - 1$, the result gives $n - 1 = (m - 1) + r$ and the required execution $t \longrightarrow^r z$. Hence $n = m + r$. The Lean proof uses constructor indices rather than natural-number subtraction, so no subtraction side conditions are needed. □

Lemma 2.8 (Uniqueness of successful terminal executions)**.** *If* $s \longrightarrow^m t$, $s \longrightarrow^n z$, $\mathsf{Final}(t)$, *and* $\mathsf{Final}(z)$, *then* $m = n$ *and* $t = z$.

Lean: `Machine.Steps.final_unique`

Proof. The prefix lemma gives $n = m + r$ and $t \longrightarrow^r z$. Finality of t makes it normal. The normal-state lemma yields $r = 0$ and $t = z$, and therefore $m = n$. □

Lemma 2.9 (A state on a successful execution is not stuck)**.** *If* $s \longrightarrow^m t$, $s \longrightarrow^n z$, *and* $\mathsf{Final}(z)$, *then* t *is not stuck.*

Lean: `Machine.Steps.not_stuck_of_final`

Proof. Suppose t were stuck. The prefix lemma gives a remaining execution $t \longrightarrow^r z$. Normality of t forces $t = z$. Since z is final, t is final, contradicting the unsuccessful part of stuckness. □

Lemma 2.10 (Removing a prescribed first transition)**.** *If* $s \to s'$ *and* $s \longrightarrow^{n+1} t$, *then* $s' \longrightarrow^n t$.

Lean: `Machine.Steps.drop_first`

Proof. The positive execution has a first transition $s \to u$ and a remainder $u \longrightarrow^n t$. Determinism identifies u with s', giving the stated remainder. The conclusion concerns the *prescribed* successor s'; without determinism it would only be valid for some successor. □

Lemma 2.11 (A stuck execution continues after an available step)**.** *If* $s \to s'$, $s \longrightarrow^n t$, *and* $\mathsf{Stuck}(t)$, *then for some* k, $s' \longrightarrow^k t$.

Lean: `Machine.Steps.stuck_after_step`

Proof. The execution cannot have length zero, since then $s = t$ would be normal despite $s \to s'$. Its first transition is therefore $s \to u$, with a remaining execution to t. Determinism gives $u = s'$. Take the length of that remainder as k. The code does not retain the additional equality $n = k + 1$, because later uses need only the existence of a remainder. □

3 Cost relations and state refinement

Definition 3.1 (Cost preorder)**.** A cost order is a reflexive, transitive relation Q on $\mathbb{N}$.

Lean: `CostOrder`

The relation and its two preorder proofs are stored together. Antisymmetry, decidability, and compatibility with addition are not part of this definition.

Definition 3.2 (Reasonable triple)**.** A triple (Q, P, S) of cost preorders is *reasonable* if all four of the following implications hold:

$$\begin{aligned} a\,Q\,b \;\wedge\; c\,Q\,d &\implies (a+c)\,Q\,(b+d), && \text{(addition)}\\ a\,P\,b &\implies b \le a, && \text{(decrease)}\\ a\,P\,b &\implies a\,S\,b, && \text{(inclusion)}\\ a\,P\,b \;\wedge\; b\,Q\,c \;\wedge\; c\,S\,d &\implies a\,Q\,d. && \text{(sandwich)} \end{aligned}$$

Lean: `Reasonable`

These four fields express the paper's three conditions: the decrease and inclusion fields are the two parts of its condition (B). The decrease field will justify a strict descent in the soundness proof. Inclusion is used when turning safety's auxiliary equivalences into an asymmetric up-to relation.

Definition 3.3 (Converse and the basic cost orders)**.** The converse preorder is $a\,Q^{-1}\,b$ if $b\,Q\,a$. The four named orders are

$$\begin{aligned} a\,E\,b &\iff a = b, \quad a\,I\,b \iff b \le a,\\ a\,H\,b &\iff a \le b, \quad a\,U\,b \iff \text{true}. \end{aligned}$$

Here E, I, H, and U denote `exact`, `improvement`, `overhead`, and `any`, respectively.

Lean: `CostOrder.converse`

Lean: `CostOrder.exact`

Lean: `CostOrder.improvement`

Lean: `CostOrder.overhead`

Lean: `CostOrder.any`

Verification of the preorder fields. Converse preserves reflexivity. Given $b\,Q\,a$ and $c\,Q\,b$, transitivity of Q gives $c\,Q\,a$, establishing transitivity of Q^{-1}. Equality is reflexive and transitive. The relation I is reflexive by $a \le a$; from $b \le a$ and $c \le b$ one obtains $c \le a$. The corresponding verification for H uses the usual order direction. All requirements for the universal relation U are immediate. □

Lemma 3.4 (Exact costs are reasonable)**.** *The triple* (E, E, E) *is reasonable.*

Lean: `CostOrder.reasonable_exact`

Proof. Substitute equals for equals in the addition condition. The decrease condition reduces to $a \le a$, inclusion is equality included in itself, and the sandwich condition is transitivity of equality. □

Lemma 3.5 (Improvement costs are reasonable)**.** *The triple* (I, I, I) *is reasonable.*

Lean: `CostOrder.reasonable_improvement`

Proof. For addition, $b \le a$ and $d \le c$ give $b + d \le a + c$. Decrease is the definition of I, and inclusion is reflexive inclusion. The sandwich premises give $d \le c \le b \le a$, hence $a\, I\, d$. □

Lemma 3.6 (Unconstrained termination costs are reasonable)**.** *The triple* (U, I, U) *is reasonable.*

Lean: `CostOrder.reasonable_termination`

Proof. Addition, inclusion, and the sandwich condition have universally true conclusions. Decrease is again the definition of I. □

Lemma 3.7 (Overhead with exact auxiliary costs is reasonable)**.** *The triple* (H, E, E) *is reasonable.*

Lean: `CostOrder.reasonable_overhead`

Proof. The addition condition is monotonicity of addition for $\le$. Equality implies the required weak decrease and includes into itself. In the sandwich condition, substitute $a = b$ and $c = d$ into $b \le c$ to obtain $a \le d$. □

Definition 3.8 (State refinement)**.** For a cost preorder Q, define

$$s \preceq_Q t \iff \begin{array}{c}\text{for every } n, z \text{ with } s \longrightarrow^n z \text{ and } \mathsf{Final}(z),\\ \text{there exist } m, w \text{ with } n\, Q\, m,\ t \longrightarrow^m w, \text{ and } \mathsf{Final}(w).\end{array}$$

State equivalence is mutual refinement: $s \simeq_Q t$ if $s \preceq_Q t$ and $t \preceq_Q s$.

Lean: `Machine.Refines`

Lean: `Machine.Equiv`

If s has no successful execution, its refinement obligation is vacuous. The definition therefore does not distinguish stuckness from divergence unless a surrounding observation supplies additional distinctions. For $Q = I$, the target must terminate in no more steps than the source. For $Q = H$, it must terminate in at least as many steps. For $Q = E$, the counts agree exactly; for $Q = U$, counts are ignored.

Lemma 3.9 (Reflexivity)**.** *For every s, $s \preceq_Q s$.*

Lean: `Machine.Refines.refl`

Proof. Given a successful execution of length n from s, choose that same execution and final state as witnesses. Reflexivity supplies $n\,Q\,n$. □

Lemma 3.10 (Transitivity)**.** *If $s \preceq_Q t$ and $t \preceq_Q v$, then $s \preceq_Q v$.*

Lean: `Machine.Refines.trans`

Proof. Take a successful execution of length n from s. The first refinement supplies a successful execution of length m from t, with $n\,Q\,m$. Apply the second refinement to this execution to obtain a successful execution of length k from v, with $m\,Q\,k$. Transitivity gives $n\,Q\,k$, and these are the required witnesses. Determinism is unnecessary. □

Lemma 3.11 (Monotonicity in the cost relation)**.** *If $Q \subseteq P$ and $s \preceq_Q t$, then $s \preceq_P t$.*

Lean: `Machine.Refines.mono`

Proof. Reuse each execution and final-state witness supplied by the original refinement. Replace its cost proof $n\,Q\,m$ by $n\,P\,m$ using the inclusion. □

4 Counting simulation and its soundness

Definition 4.1 (Up-to closure)**.** Given a relation R on states and cost orders P, S, define

$$\mathsf{UpTo}_{P,S}(R)(s,t) \iff \exists u, v\ (s \preceq_P u \ \wedge\ R(u,v) \ \wedge\ v \preceq_S t).$$

Lean: `Machine.UpTo`

The witnesses occur in path order. In particular the right-hand refinement is from v to t, not from t to v. Auxiliary refinement can change the states at which the candidate relation is resumed.

Definition 4.2 (Counting simulation)**.** A relation R is a (Q, P, S)-counting simulation if whenever $R(s,t)$:

(A) If s is final, then t is final.

(B) For every transition $s \to s'$, one of the following holds:

(i) There exist k, z such that $s' \longrightarrow^k z$ and z is stuck.

(ii) There exist k, l, h_1, h_2 such that

$$s' \longrightarrow^k h_1, \qquad t \longrightarrow^l h_2, \qquad (1+k)\,Q\,l, \qquad \mathsf{UpTo}_{P,S}(R)(h_1, h_2).$$

Lean: `Machine.CountingSimulation`

The leading 1 charges for the already prescribed transition $s \to s'$. Neither k nor l is required to be positive. The stuck alternative is a deliberate part of the definition, consistent with observing only successful termination. The finality condition is stronger than simply allowing the target to reach a final state after some extra steps: it requires the target to be final already when the source is final.

Theorem 4.3 (Soundness of counting simulation)**.** *Suppose M is deterministic, (Q, P, S) is reasonable, and R is a (Q, P, S)-counting simulation. Then*

$$R(s,t) \implies s \preceq_Q t.$$

Lean: `Machine.CountingSimulation.sound`

This is the abstract transition-system form of the paper's Proposition 37. Its proof is a strong induction on the length of a successful source execution. Although simulation provides a step-wise, coinductive style of reasoning, the soundness theorem itself is proved by induction on a finite observation; no coinductive datatype is introduced in this file.

Proof. We prove, simultaneously for all states s, t, z, that if $R(s,t)$, $s \longrightarrow^N z$, and $\mathsf{Final}(z)$, then there are m, w such that $N\,Q\,m$, $t \longrightarrow^m w$, and $\mathsf{Final}(w)$. Use strong induction on N; the induction hypothesis applies to every smaller natural number and every pair of related states.

Base case. If $N = 0$, then $s = z$ is final. Condition (A) makes t final. Choose the zero-step execution at t and use $0\,Q\,0$.

Positive case. Write $N = n + 1$ and decompose the successful execution:

$$s \to s' \longrightarrow^n z.$$

Apply condition (B) of simulation. Its stuck alternative would give a finite execution from s' to a stuck state. This is impossible by the lemma that a state on a successful execution cannot be stuck, applied to the successful execution $s' \longrightarrow^n z$.

Consequently there are k, l, h_1, h_2, u, v satisfying

$$\begin{gathered} s' \longrightarrow^k h_1, \qquad t \longrightarrow^l h_2, \qquad (1+k)\,Q\,l, \\ h_1 \preceq_P u, \qquad R(u,v), \qquad v \preceq_S h_2. \end{gathered}$$

The prefix lemma, using determinism and finality of z, gives a remainder length r such that

$$n = k + r, \qquad h_1 \longrightarrow^r z.$$

Apply $h_1 \preceq_P u$ to this remaining successful execution. It supplies a, z_a with

$$r\,P\,a, \qquad u \longrightarrow^a z_a, \qquad \mathsf{Final}(z_a).$$

The decrease condition of reasonableness gives $a \le r$. Since $N = 1 + k + r$, we have $a < N$. This is the strict inequality needed to invoke the strong induction hypothesis. The proof would not be justified for arbitrary auxiliary cost preorders.

Applying that hypothesis to $R(u,v)$ gives b, z_b such that

$$a\,Q\,b, \qquad v \longrightarrow^b z_b, \qquad \mathsf{Final}(z_b).$$

Now apply $v \preceq_S h_2$. We obtain c, z_c with

$$b\,S\,c, \qquad h_2 \longrightarrow^c z_c, \qquad \mathsf{Final}(z_c).$$

The sandwich condition combines $r\,P\,a$, $a\,Q\,b$, and $b\,S\,c$ to yield $r\,Q\,c$. Additivity of Q then gives

$$(1 + k + r)\,Q\,(l + c), \quad \text{that is,} \quad N\,Q\,(l + c).$$

Concatenate $t \longrightarrow^l h_2$ with $h_2 \longrightarrow^c z_c$. The result is a successful execution of length $l + c$ from t, which supplies the required witnesses. □

The code follows these witnesses exactly: its `count` is $n = k + r$, `smaller` is $a < n + 1$, `rc` is $r\,Q\,c$, and `total` is $(n + 1)\,Q\,(l + c)$. The inclusion $P \subseteq S$ is not needed in this particular proof; it is part of the shared reasonable triple and will be used by the specimen argument.

5 Typed observations and contextual lifting

Definition 5.1 (Observation interface)**.** Let J be a type of interfaces, and for each $i \in J$ let F_i be a type of fragments and C_i a type of contexts. An observation interface consists of a predicate $\mathsf{Allowed}(c)$ for contexts and a relation

$$\mathsf{Obs}(c, a, s), \qquad c \in C_i,\ a \in F_i,\ s \in X.$$

The intended reading is that placing a in c and initializing the result can produce state s.

Lean: `Observations`

The common index i prevents a fragment and a context of incompatible interfaces from being compared. Observation is a relation rather than a total function. It can be undefined, and the structure does not require uniqueness of the produced state. The current code does not yet define it by graph plugging.

Definition 5.2 (Contextual refinement and equivalence)**.** For $a, b \in F_i$, write $a \preceq_Q^{\mathcal{O}} b$ if for every $c \in C_i$ and states s, t,

$$\mathsf{Allowed}(c) \ \wedge\ \mathsf{Obs}(c, a, s) \ \wedge\ \mathsf{Obs}(c, b, t) \implies s \preceq_Q t.$$

Contextual equivalence $a \simeq_Q^{\mathcal{O}} b$ is the conjunction of this refinement and the refinement in the opposite direction.

Lean: `Observations.Refines`

Lean: `Observations.Equiv`

The quantifiers compare every pair of observations of the two fragments in the same allowed context. The definition imposes no existence obligation when either fragment has no observation there. Also, state-refinement reflexivity alone does not establish contextual reflexivity for this fully general interface: two observations of the same fragment might give different states. A concrete, functional initialization relation can supply the missing property, but it is not assumed here.

Proposition 5.3 (Contextual refinement from a lifted relation)**.** *Let T_i be a relation on F_i for each interface i, and let L be a relation on states. Suppose:*

1. *Whenever $T_i(a, b)$ and c is allowed, observations $\mathsf{Obs}(c, a, s)$ and $\mathsf{Obs}(c, b, t)$ imply $L(s, t)$.*
2. *$L(s, t)$ implies $s \preceq_Q t$.*

Then $T_i(a, b)$ implies $a \preceq_Q^{\mathcal{O}} b$.

Lean: `Observations.contextual_of_lifting`

Proof. Fix $T_i(a, b)$, an allowed c, and the two observations. The first hypothesis, named `initial_lift` in Lean, puts their initial states in L. The second hypothesis, named `sound`, then gives the state refinement required by the definition of contextual refinement. □

This isolates the logical step of the paper's Proposition 38. Its concrete proof still has to establish the initial-specimen hypothesis for actual plugging and initialization. The proposition above neither asserts nor requires closure under plugging at this abstract level.

Proposition 5.4 (Contextual equivalence from a lifted relation)**.** *Under the same initial-lifting hypothesis, if $L(s,t)$ implies $s \simeq_Q t$, then $T_i(a,b)$ implies $a \simeq_Q^{\mathcal{O}} b$.*

Lean: `Observations.contextual_equiv_of_lifting`

Proof. For the forward refinement, apply the preceding proposition using the forward component of state equivalence. For the reverse refinement, fix observations of b and a in an allowed context. Put them into `initial_lift` in the order a, b, obtaining L from the state of a to the state of b. Use the reverse component of its state equivalence. This does not require that T_i or L be symmetric. □

6 The abstract specimen interface

Definition 6.1 (Focus modes)**.** The type of modes has exactly three constructors: search, backtrack, and rewrite.

Lean: `Focus`

They represent the paper's searching, backtracking, and triggering focuses. The derived decidable equality allows their distinctness to be checked by finite case analysis. This datatype records only a mode, not a graph position or a focus edge.

Definition 6.2 (Specimen system)**.** A specimen system G over M consists of a type Σ of specimens, maps $\ell, r : \Sigma \to X$, a mode map $\mathsf{mode} : \Sigma \to \mathsf{Focus}$, a predicate Enter on specimens, and a predicate Root on states. Its laws are:

1. *Rootedness preservation:* $\mathsf{Root}(s)$ and $s \to t$ imply $\mathsf{Root}(t)$.
2. *Final-state agreement:* for every $c \in \Sigma$, $\mathsf{Final}(\ell(c)) \iff \mathsf{Final}(r(c))$.
3. *Context-only traversal:* suppose both endpoints of c are rooted and c is either backtracking or searching without entering. If either endpoint has an outgoing transition, there is a specimen d whose focus is remote and such that

$$\ell(c) \to \ell(d), \qquad r(c) \to r(d).$$

The terms "rooted," "entering," and "remote" receive their precise abstract meanings from these data and the definitions immediately below.

Lean: `SpecimenSystem`

The corresponding fields are `Specimen`, `left`, `right`, `mode`, `entering`, `rooted`, `rooted_step`, `same_final`, and `context_step`. The last three are hypotheses needed to construct an instance, not results proved here from a graph model. The intended specimen consists of a multi-hole focussed context and two pointwise related vectors of fragments. That intended representation is not a field of this abstract structure.

In particular, the interface does not identify modes with a focus field on states, define rootedness as reachability from an initial state, or define entering by incidence with a hole. The context-traversal law captures the consequence of graph locality and output-closure that the simulation proof uses. Its availability premise is symmetric between the two endpoints, which will be essential in the converse proof.

Definition 6.3 (Remote, settled, and lifted specimens)**.** A specimen c is remote if

$$\mathsf{Remote}(c) \iff \mathsf{mode}(c) \neq \mathsf{rewrite} \ \vee \ \neg\mathsf{Enter}(c).$$

It is settled if

$$\mathsf{Settled}(c) \iff \mathsf{mode}(c) = \mathsf{backtrack} \ \vee \ (\mathsf{mode}(c) = \mathsf{search} \wedge \neg\mathsf{Enter}(c)).$$

Its lifted state relation is

$$\mathcal{L}_G(s,t) \iff \exists c \in \Sigma \ \mathsf{Remote}(c) \wedge \mathsf{Root}(\ell(c)) \wedge \mathsf{Root}(r(c)) \wedge s = \ell(c) \wedge t = r(c).$$

Lean: `SpecimenSystem.Remote`

Lean: `SpecimenSystem.Settled`

Lean: `SpecimenSystem.Lifting`

Remote excludes only an entering rewrite focus. It allows an entering search focus. Settled is stronger: it excludes every rewrite focus and also excludes an entering search focus. The two equalities in the lifting definition expose a particular specimen as a witness for the endpoints.

Lemma 6.4 (Settled implies remote)**.** *If* $\mathsf{Settled}(c)$*, then* $\mathsf{Remote}(c)$*.*

Lean: `SpecimenSystem.settled_remote`

Proof. A backtracking or searching mode is distinct from the rewrite mode. Therefore the first disjunct in the remote predicate holds in either case. □

Lemma 6.5 (Rootedness along a finite execution)**.** *If* $\mathsf{Root}(s)$ *and* $s \longrightarrow^n t$*, then* $\mathsf{Root}(t)$*.*

Lean: `SpecimenSystem.rooted_steps`

Proof. Induct on the execution. At length zero nothing changes. For a positive execution, the one-step rootedness law makes its first successor rooted; the induction hypothesis propagates this property through the remainder. □

Definition 6.6 (Abstract non-rewrite quasi-specimen)**.** For relations A, B on states, define $\mathsf{Quasi}_G(A,B)(s,t)$ to mean that *if* s and t are rooted, then there is a specimen c with

$$\mathsf{mode}(c) \neq \mathsf{rewrite}, \qquad \mathsf{Root}(\ell(c)), \quad \mathsf{Root}(r(c)),$$
$$A(s,\ell(c)), \qquad B(r(c),t).$$

Lean: `SpecimenSystem.Quasi`

The outer implications matter: no witness is required if either endpoint is unrooted. Subsequent proofs establish both rootedness hypotheses before using a quasi-specimen. Unlike the paper's full definition, this abstract consequence does not compare focus labels of s, t with that of c; there is no state-focus map in the interface. It specifically packages the non-rewrite situation used by safety and robustness.

Lemma 6.7 (A rooted quasi-specimen supplies an up-to witness)**.** *If* $\mathsf{Quasi}_G(\preceq_P, \preceq_S)(s,t)$ *and both* s, t *are rooted, then* $\mathsf{UpTo}_{P,S}(\mathcal{L}_G)(s,t)$*.*

Lean: `SpecimenSystem.quasi_upTo`

Proof. Obtain its witness c. Its non-rewrite mode makes it remote. Together with rootedness of its endpoints, this gives $\mathcal{L}_G(\ell(c), r(c))$. Use $\ell(c)$ and $r(c)$ as the intermediate states in the up-to definition, with the two refinements furnished by the quasi-specimen. □

Definition 6.8 (Input-safety)**.** The predicate $\mathsf{InputSafe}_G(Q, P)$ requires the following for every specimen c whose endpoints are rooted, whose mode is search, and which is entering. At least one of these three cases must hold:

(I) Both endpoints have finite executions to stuck states. Their counts may be unrelated, and either count may be zero.

(II) For some settled specimen d and $k, l \in \mathbb{N}$,

$$\ell(c) \longrightarrow^{1+k} \ell(d), \quad r(c) \longrightarrow^{l} r(d), \quad (1+k)\, Q\, l.$$

(III) For some states a, b and $k, l \in \mathbb{N}$,

$$\ell(c) \longrightarrow^{1+k} a, \quad r(c) \longrightarrow^{1+l} b, \quad (1+k)\, Q\, (1+l),$$

and $\mathsf{Quasi}_G(\simeq_P, \simeq_P)(a, b)$ holds.

Lean: `SpecimenSystem.InputSafe`

Case II requires a positive left execution but permits zero right steps. Case III requires positive executions on both sides and auxiliary *equivalences*. Retaining both directions of those equivalences is necessary for reusing safety in the converse theorem.

Definition 6.9 (Robustness)**.** The predicate $\mathsf{Robust}_G(Q, P, S)$ requires that for every specimen c with rooted endpoints, rewrite mode, and $\neg\mathsf{Enter}(c)$, each transition $\ell(c) \to s'$ satisfies one of:

(i) s' has a finite execution to a stuck state;

(ii) for some k, l, a, b,

$$s' \longrightarrow^{k} a, \quad r(c) \longrightarrow^{l} b, \quad (1+k)\, Q\, l,$$

and $\mathsf{Quasi}_G(\preceq_P, \preceq_S)(a, b)$ holds.

Lean: `SpecimenSystem.Robust`

This is a rooted, remote restriction of the consequences used from the paper's robustness condition. The paper's alternative that an endpoint is unrooted is contradictory under the present premises. The abstract machine does not classify its transition relation into search and rewrite rules; it is the specimen's mode that determines where this predicate is invoked. Establishing the correspondence with concrete rewrite transitions is part of the missing hypernet instance.

7 The forward robustness argument

Throughout this section M is deterministic, (Q, P, S) is reasonable, and G is a specimen system satisfying $\mathsf{InputSafe}_G(Q, P)$ and $\mathsf{Robust}_G(Q, P, S)$. In particular, all the interface laws of Definition 6.2 are available.

Theorem 7.1 (Forward construction of a counting simulation)**.** *The relation $\mathcal{L}_G$ is a (Q, P, S)-counting simulation.*

Lean: `SpecimenSystem.countingSimulation`

This is an abstract form of the forward clause of the paper's Proposition 40. The proof checks the two simulation fields directly.

Proof. Take $\mathcal{L}_G(s, t)$ and choose a witnessing specimen c. Thus $s = \ell(c)$, $t = r(c)$, the specimen is remote, and both endpoints are rooted.

Finality. If s is final, final-state agreement for c makes t final. This verifies simulation condition (A).

A common context-traversal case. Suppose c is settled and there is a prescribed transition $s \to s'$. The context-traversal law gives a remote specimen d and transitions

$$\ell(c) \to \ell(d), \qquad r(c) \to r(d).$$

Determinism identifies s' with $\ell(d)$. Both new endpoints are rooted, so $\mathcal{L}_G(\ell(d), r(d))$. The simulation matching case can use zero further left steps and one right step. Its cost obligation is $(1 + 0)\, Q\, 1$, which is reflexivity of Q. To obtain the up-to relation, choose $\ell(d), r(d)$ as intermediate states and insert reflexive refinements for P and S on either side of their lifted relation. These are the local proof witnesses named `context_case` and `settled_case` in Lean.

It remains to cover all possible modes of c.

Backtracking. Such a specimen is settled by definition, so the common context-traversal argument applies.

Searching without entering. This is the other settled case and uses the same argument.

Entering search. Apply input-safety, obtaining one of its three cases.

(I) There is a stuck execution from $\ell(c)$. Since $\ell(c) \to s'$ is available, the stuck-after-step lemma removes that first step and gives a stuck execution from s'. This is the stuck alternative of simulation. Safety also supplies a stuck execution on the right, but the forward simulation condition does not need it.

(II) There are d, k, l with d settled,

$$\ell(c) \longrightarrow^{1+k} \ell(d), \qquad r(c) \longrightarrow^{l} r(d), \qquad (1 + k)\, Q\, l.$$

Remove the prescribed first transition from the left execution using determinism. The resulting execution from s' to $\ell(d)$ has length k. Rootedness propagates to both endpoints of d, and settledness implies remoteness. Hence $\mathcal{L}_G(\ell(d), r(d))$. Add reflexive auxiliary refinements as in the context-traversal case. The matching counts are k, l, with the cost condition already supplied by safety.

(III) There are k, l, a, b with the positive executions and quasi-specimen of safety III. Remove the prescribed first step from the left execution, giving $s' \longrightarrow^{k} a$; retain the right

execution of length $1 + l$. Both a and b are rooted. Unpack the quasi-specimen to get a non-rewrite specimen d with rooted endpoints and

$$a \simeq_P \ell(d), \qquad r(d) \simeq_P b.$$

Use the forward half of the first equivalence to obtain $a \preceq_P \ell(d)$. The forward half of the second gives $r(d) \preceq_P b$, and monotonicity with $P \subseteq S$ changes this to $r(d) \preceq_S b$. Thus the quasi-up-to lemma supplies $\mathsf{UpTo}_{P,S}(\mathcal{L}_G)(a, b)$. The matching counts are $k, 1 + l$, and their required cost relation is exactly $(1 + k)\, Q\, (1 + l)$.

Rewrite mode. Remoteness now implies $\neg\mathsf{Enter}(c)$, since the other remote disjunct contradicts the current mode. Apply robustness to the prescribed transition $\ell(c) \to s'$. If it provides a stuck execution from s', use the stuck alternative of simulation. Otherwise it provides matching executions of lengths k, l, the cost proof $(1+k)\, Q\, l$, and a quasi-specimen up to $\preceq_P, \preceq_S$. Its left endpoint is rooted by the prescribed step followed by the k-step execution; its right endpoint is rooted by its l-step execution. The quasi-up-to lemma therefore supplies the required up-to relation. This completes every mode and every simulation condition. □

Theorem 7.2 (Forward state sufficiency)**.** *Under the hypotheses of this section,*

$$\mathcal{L}_G(s, t) \implies s \preceq_Q t.$$

Lean: `SpecimenSystem.sufficiency`

Proof. The preceding theorem constructs a counting simulation on $\mathcal{L}_G$. Apply counting-simulation soundness, Theorem 4.3, with the same deterministic machine and reasonable triple. □

All graph-specific information in this theorem is supplied through the specimen system and the safety and robustness premises. The theorem's short proof is therefore not evidence that the missing graph-locality verification is automatic.

8 Opposite specimens and the converse argument

Definition 8.1 (Opposite specimen system)**.** The opposite system G^{op} has the same specimens, modes, entering predicate, and rooted-state predicate as G. It swaps the two endpoint maps:

$$\ell_{G^{\mathrm{op}}}(c) = r_G(c), \qquad r_{G^{\mathrm{op}}}(c) = \ell_G(c).$$

Lean: `SpecimenSystem.opposite`

Verification of its interface laws. Rootedness preservation is unchanged. Final-state agreement is the reverse implication pair of the original agreement. For context-only traversal, exchange the two rootedness hypotheses and the two alternatives of the availability premise. Apply the original law to get a remote specimen d and two transitions. Return those transitions in the opposite order. This explains why the interface law permits availability on either side, rather than only on the left. □

Lemma 8.2 (Lifting commutes with taking the opposite)**.** *For all states s, t,*

$$\mathcal{L}_{G^{\mathrm{op}}}(s,t) \iff \mathcal{L}_G(t,s).$$

Lean: `SpecimenSystem.lifting_opposite`

Proof. In either direction retain the witnessing specimen and its remoteness. Exchange the two rootedness proofs and the two endpoint equalities. The resulting tuple is precisely the witness required by the other lifting definition. □

Theorem 8.3 (Converse construction of a counting simulation)**.** *Suppose M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q^{-1}, P)$ holds, and G^{op} is robust for (Q, P, S). Then $\mathcal{L}_{G^{\mathrm{op}}}$ is a (Q, P, S)-counting simulation.*

Lean: `SpecimenSystem.countingSimulation_converse`

The safety premise is a property of the *original* system at the converse cost relation. It is not assumed that the opposite system already satisfies input-safety. This distinction is important when the right execution in safety II has length zero.

Proof. Choose a witness c of $\mathcal{L}_{G^{\mathrm{op}}}(s,t)$, so $s = r(c)$, $t = \ell(c)$, c is remote, and both endpoints are rooted. Finality of $r(c)$ implies finality of $\ell(c)$ by the reverse direction of final-state agreement. This establishes condition (A). For condition (B), fix a transition $r(c) \to s'$.

Settled case. If c is settled, the symmetric availability premise of context-only traversal applies to this right transition. Obtain a remote specimen d with transitions

$$\ell(c) \to \ell(d), \qquad r(c) \to r(d).$$

Determinism gives $s' = r(d)$. Propagate rootedness, and obtain $\mathcal{L}_{G^{\mathrm{op}}}(r(d), \ell(d))$. Choose zero further steps from s' and one step from $\ell(c)$. The cost condition is $1\, Q\, 1$, and reflexive refinements give the up-to witness. This covers backtracking and non-entering search.

Entering search. Input-safety for Q^{-1}, P supplies three cases, which must be read in the original endpoint order.

(I) Use the supplied stuck execution from $r(c)$. The prescribed step $r(c) \to s'$ and the stuck-after-step lemma give a stuck execution from s'. The supplied stuck execution from $\ell(c)$ is not needed in this direction.

(II) There are a settled specimen d and counts k, l with

$$\ell(c) \longrightarrow^{1+k} \ell(d), \qquad r(c) \longrightarrow^{l} r(d), \qquad l\, Q\, (1+k).$$

Both endpoints of d are rooted. We distinguish the two possible forms of l.

If $l = j+1$, remove the prescribed transition from the right execution to get $s' \longrightarrow^{j} r(d)$. Keep the left execution of length $1+k$. The opposite lifted pair is $(r(d), \ell(d))$, and its cost condition is $(1+j)\, Q\, (1+k)$, exactly the supplied relation with $l = j+1$. Reflexive auxiliary refinements finish this subcase.

If $l = 0$, the zero-step equality gives $r(c) = r(d)$. Thus the prescribed transition is an outgoing transition from $r(d)$. Apply context-only traversal to the *settled* specimen d. Obtain a remote specimen e and transitions

$$\ell(d) \to \ell(e), \qquad r(d) \to r(e).$$

Determinism identifies s' with $r(e)$. Propagating rootedness gives the opposite lifted pair $(r(e), \ell(e))$. There are zero further source steps, whereas the target execution is

$$\ell(c) \longrightarrow^{1+k} \ell(d) \to \ell(e),$$

of length $(1+k)+1$. Its cost obligation is $1\,Q\,((1+k)+1)$. The safety relation in this subcase is $0\,Q\,(1+k)$. Add $1\,Q\,1$ using additivity of Q to obtain the required relation. This additional traversal is the essential zero-step correction; the proof never attempts to remove a step from a zero-step execution.

(III) There are k, l, a, b with

$$\ell(c) \longrightarrow^{1+k} a, \qquad r(c) \longrightarrow^{1+l} b, \qquad (1+l)\,Q\,(1+k),$$

and a quasi-specimen for auxiliary equivalences. Remove the prescribed right transition to get $s' \longrightarrow^{l} b$. Both a, b are rooted; unpacking the quasi-specimen yields a non-rewrite specimen d with rooted endpoints and

$$a \simeq_P \ell(d), \qquad r(d) \simeq_P b.$$

This time take the *reverse* halves: $b \preceq_P r(d)$ and $\ell(d) \preceq_P a$. Enlarge the latter auxiliary order using $P \subseteq S$ to obtain $\ell(d) \preceq_S a$. Since $\mathcal{L}_{G^{\mathrm{op}}}(r(d), \ell(d))$, these form

$$\mathsf{UpTo}_{P,S}(\mathcal{L}_{G^{\mathrm{op}}})(b, a).$$

The matching counts are $l, 1+k$, with the already supplied cost relation $(1+l)\,Q\,(1+k)$.

Rewrite mode. Remoteness gives $\neg\mathsf{Enter}(c)$. Apply robustness in G^{op} to the prescribed transition from $r(c)$, which is its left endpoint. The stuck alternative is already the required one. Otherwise the two matching executions and their cost relation are supplied directly. Rootedness propagates along the prescribed transition and both executions. Apply the quasi-up-to lemma *in the opposite system* to obtain the up-to relation on $\mathcal{L}_{G^{\mathrm{op}}}$. This completes the proof. □

9 State and contextual consequences

The following results compose the two simulation constructions with the soundness and observation lemmas. They are short in Lean because their mathematical work has been isolated in those earlier results.

Theorem 9.1 (Converse state sufficiency)**.** *If M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q^{-1}, P)$ holds, and G^{op} is robust for (Q, P, S), then*

$$\mathcal{L}_G(s, t) \implies t \preceq_Q s.$$

Lean: `SpecimenSystem.sufficiency_converse`

Proof. The lifting-opposite lemma changes the premise into $\mathcal{L}_{G^{\mathrm{op}}}(t, s)$. The converse construction supplies a counting simulation on this relation. Apply counting-simulation soundness to obtain the stated refinement. □

Theorem 9.2 (Mutual state refinement)**.** *Suppose M is deterministic, (Q, P, S) is reasonable, G is input-safe for both (Q, P) and (Q^{-1}, P), and both G and G^{op} are robust for (Q, P, S). Then*

$$\mathcal{L}_G(s, t) \implies s \simeq_Q t.$$

Lean: `SpecimenSystem.equivalence`

Proof. Forward state sufficiency supplies $s \preceq_Q t$. Converse state sufficiency supplies $t \preceq_Q s$. These are exactly the two components of state equivalence. □

The main cost relation Q itself need not be symmetric. The two directional refinement statements use the same Q, with source and target counts exchanged when the states are exchanged.

For the contextual results, fix the observation interface of the earlier section and a family T_i of relations on fragments. Assume the following *initial-lifting property*: for $T_i(a, b)$, every allowed context c and observations $\mathsf{Obs}(c, a, s)$, $\mathsf{Obs}(c, b, t)$ give $\mathcal{L}_G(s, t)$. This property is the explicit premise `initial_lift` in the source. It connects the abstract specimen system with the separately supplied observation relation.

Theorem 9.3 (Forward contextual sufficiency)**.** *If M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q, P)$ and $\mathsf{Robust}_G(Q, P, S)$ hold, and the initial-lifting property holds, then*

$$T_i(a, b) \implies a \preceq_Q^{\mathcal{O}} b.$$

Lean: `SpecimenSystem.contextual_sufficiency`

Proof. Use contextual refinement from a lifted relation, taking $L = \mathcal{L}_G$. Its initial-lifting premise is precisely the one assumed here. Its state-soundness premise is forward state sufficiency. Applying that proposition to $T_i(a, b)$ gives the desired contextual refinement. □

Theorem 9.4 (Converse contextual sufficiency)**.** *If M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q^{-1}, P)$ holds, G^{op} is robust for (Q, P, S), and the same initial-lifting property holds, then*

$$T_i(a, b) \implies b \preceq_Q^{\mathcal{O}} a.$$

Lean: `SpecimenSystem.contextual_sufficiency_converse`

Proof. Fix $T_i(a, b)$, an allowed context, an observation of b producing s, and an observation of a producing t. Supply the observations to initial lifting in the original a, b order to get $\mathcal{L}_G(t, s)$. Converse state sufficiency then gives $s \preceq_Q t$, the required contextual comparison. □

These are conditional abstract counterparts of the two clauses of the paper's Theorem 6. Refocusing, output-closure, type-correct plugging, and the paper's concrete specimen definitions have not disappeared from the mathematics: their required consequences are currently represented by the specimen-system laws and initial-lifting premise. Instantiating those hypotheses is the remaining concrete formalization problem.

10 Semantic examples and regression proofs

The examples are small abstract machines used to check the meaning and edge cases of the definitions. They do not formalize the paper's lambda-calculus translation, copying UAM, or program-equivalence examples.

10.1 A countdown machine

Definition 10.1 (Countdown)**.** The countdown machine has state set $\mathbb{N}$, transition $s \to t$ if $s = t + 1$, and final predicate $s = 0$.

Lean: `Examples.countdown`

Verification of the machine law. If $s = 0$, an outgoing transition would imply $0 = t + 1$, impossible for a natural number t. Thus final states are normal as required by the machine structure. □

Lemma 10.2 (Countdown is deterministic)**.** *If $s \to a$ and $s \to b$, then $a = b$.*

Lean: `Examples.countdown_deterministic`

Proof. The premises give $s = a + 1$ and $s = b + 1$. Cancellation of successor gives $a = b$. □

Lemma 10.3 (Canonical countdown execution)**.** *For every $n \in \mathbb{N}$, the countdown machine has $n \longrightarrow^n 0$.*

Lean: `Examples.countdown_run`

Proof. Induct on n. For $n = 0$, use the zero-step constructor. For $n + 1$, take the transition $n + 1 \to n$ and prepend it to the induction hypothesis $n \longrightarrow^n 0$. □

Proposition 10.4 (Countdown characterizes the cost relation)**.** *For every cost preorder Q and states $a, b \in \mathbb{N}$,*

$$a \preceq_Q b \quad \Longleftrightarrow \quad a \, Q \, b.$$

Lean: `Examples.countdown_refines_iff`

Proof. Suppose $a \preceq_Q b$. Apply it to the canonical successful execution $a \longrightarrow^a 0$. It gives a successful execution of some length m from b and $a \, Q \, m$. Compare this execution with $b \longrightarrow^b 0$. Determinism and final-execution uniqueness give $m = b$, so $a \, Q \, b$.

Conversely assume $a \, Q \, b$. For any successful execution of length n from a, compare it with $a \longrightarrow^a 0$. Uniqueness gives $n = a$. Choose $b \longrightarrow^b 0$ as the target execution; its cost condition is $n \, Q \, b$, obtained by replacing n with a. □

The file then proves three anonymous examples, each by the proposition and the indicated numerical fact:

1. $5 \preceq_I 3$, because $3 \leq 5$.
2. $3 \not\preceq_I 5$, because $5 \not\leq 3$.

3. $5 \not\preceq_E 3$, because $5 \neq 3$.

Lean: `Examples: improvement from 5 to 3`

Lean: `Examples: no improvement from 3 to 5`

Lean: `Examples: unequal exact counts`

These verify that improvement uses the target count as the smaller quantity. Reversing the intended cost orientation would invalidate the first two examples.

10.2 A crash and an infinite loop

Definition 10.5 (Fault states and the fault machine)**.** There are four distinct states:

$$\mathsf{done}, \quad \mathsf{crash}, \quad \mathsf{stuck}, \quad \mathsf{loop}.$$

The only transitions are

$$\mathsf{crash} \to \mathsf{stuck}, \qquad \mathsf{loop} \to \mathsf{loop}.$$

Only done is final.

Lean: `Examples.FaultState`

Lean: `Examples.fault`

The four names are constructors of a finite datatype with derived decidable equality. Here the constructor stuck names a particular state; $\mathsf{Stuck}(s)$ remains the general predicate defined earlier.

Verification of the machine law. No transition has source done, because both possible source constructors are distinct from it. Consequently a final state has no outgoing transition. □

Lemma 10.6 (The fault machine is deterministic)**.** *Any two transitions with the same source have the same target.*

Lean: `Examples.fault_deterministic`

Proof. Case-analyze the two transition proofs. If both use the crash rule, both targets are stuck. If both use the loop rule, both targets are loop. Either mixed case would equate crash with loop and is impossible. □

Definition 10.7 (The crash–loop candidate)**.** Let $R(s,t)$ mean $s = \mathsf{crash}$ and $t = \mathsf{loop}$.

Lean: `Examples.crashLoop`

Lemma 10.8 (A simulation using the stuck alternative)**.** *The crash–loop candidate is an* (E,E,E)*-counting simulation.*

Lean: `Examples.crash_loop_simulation`

Proof. For its only related pair, the source is crash, which is not final; hence the finality condition is vacuous. Its only successor is stuck. Use the zero-step execution from that successor to itself in the stuck alternative of simulation. It is normal because neither transition rule starts there, and it is not final because it differs from done. □

The next anonymous example concludes $\mathsf{crash} \preceq_E \mathsf{loop}$.

Lean: `Examples: crash refines loop`

Proof. Apply counting-simulation soundness to the preceding simulation, fault determinism, the reasonable triple (E,E,E), and the only pair of R. □

There is no requirement to match a successful source execution, because none exists. Thus the result is consistent even with exact costs. The example exercises the stuck branch of the implemented proof; it does not prove that every conceivable up-to simulation proof of this same refinement must use that branch.

10.3 The zero-step converse case

Definition 10.9 (Visit specimens and endpoint maps)**.** The type of specimens has three distinct constructors entering, settled, finished, with derived decidable equality. The endpoint maps into the countdown machine are

$$\begin{aligned} \ell(\mathsf{entering}) &= 2, \quad r(\mathsf{entering}) = 1, \\ \ell(\mathsf{settled}) &= 1, \quad r(\mathsf{settled}) = 1, \\ \ell(\mathsf{finished}) &= 0, \quad r(\mathsf{finished}) = 0. \end{aligned}$$

Lean: `Examples.Visit`

Lean: `Examples.visitLeft`

Lean: `Examples.visitRight`

Definition 10.10 (Visit specimen system)**.** Every visit specimen has search mode. The entering predicate holds exactly of the constructor entering. Every state is declared rooted. Together with the endpoint maps above, these data define a specimen system over countdown.

Lean: `Examples.visitSystem`

Verification of the interface laws. Rootedness preservation is immediate since rootedness is the true predicate. Final-state agreement holds because the endpoint pairs are $(2,1)$, $(1,1)$, and $(0,0)$: only in the third pair is either endpoint zero, and then both are.

For context-only traversal, consider the specimen. The constructor entering cannot satisfy the settled premise: its mode is search and its entering predicate holds. At the constructor settled, both endpoints are 1. Choose finished and the countdown transitions $1 \to 0$ on both sides; its search mode is remote. At finished, both endpoints are zero and neither has an outgoing transition, contradicting the availability premise. These cases exhaust the datatype. □

This is a deliberately abstract instance. For example, all its modes are search, including that of the pair of final states. The specimen interface does not enforce a concrete UAM convention for final focus modes.

Lemma 10.11 (Visit input-safety)**.** *The visit system is input-safe for* (H^{-1}, E).

Lean: `Examples.visit_safe`

Proof. The entering premise forces the input specimen to be entering. Use safety II with output specimen settled and $k = l = 0$. The left execution is the single countdown transition $2 \to 1$; the right execution has zero steps at 1. The output specimen is searching without entering and is therefore settled. The cost requirement is $1\, H^{-1}\, 0$, or $0 \leq 1$. □

Lemma 10.12 (Visit converse robustness)**.** *The opposite visit system is robust for* (H, E, E).

Lean: `Examples.visit_converse_robust`

Proof. Its mode is always search. The rewrite-mode premise of robustness is therefore impossible, so every required implication holds vacuously. □

Proposition 10.13 (The converse zero-step regression)**.** *In countdown,* $1 \preceq_H 2$.

Lean: `Examples.converse_zero_step_case`

Proof. Use converse state sufficiency for the visit system. Countdown is deterministic, (H, E, E) is reasonable, and the preceding two lemmas supply its safety and converse robustness premises. The entering specimen is remote, both its endpoints are rooted, and it witnesses $\mathcal{L}_G(2,1)$. The theorem consequently gives $1 \preceq_H 2$.

In the underlying converse simulation construction, safety II provides the pair $(1,1)$ after one original left step and zero original right steps. The prescribed right step is $1 \to 0$. Context traversal at $(1,1)$ supplies $(0,0)$ and adds one transition to the original left execution, now $2 \to 1 \to 0$. Thus the source terminates in one step and the target in two, satisfying $1\, H\, 2$. This is precisely the zero-step subcase proved in the converse argument. □

11 Finite carriers and concrete interfaced graphs

We now turn to actual graph objects. The constructions in the following sections prove their own incidence and typing obligations. They do not assume the specimen laws used in the preceding abstract argument. We work with small finite vertex and edge carriers; operation labels may live in a larger universe.

11.1 Enumerations and bijections

Definition 11.1 (Finite enumeration)**.** An enumeration of a type A consists of a finite list l and a proof that every $a \in A$ belongs to l. Repetitions are allowed. This witnesses finiteness without requiring decidable equality or a canonical choice of names.

Lean: `Enumeration`

The empty type is enumerated by the empty list: completeness follows because there is no element to check. The finite ordinal $\{0, \ldots, n-1\}$ is enumerated in increasing order by Lean's finite range; its library membership theorem proves completeness, including when $n = 0$.

Lean: `Enumeration.empty`

Lean: `Enumeration.fin`

For a disjoint union $A + B$, concatenate the tagged enumerations of A and B. An element has exactly one tag, and completeness of the corresponding list places it in the appropriate half of the concatenation.

Lean: `Enumeration.sum`

For a subset $\{a \in A \mid p(a)\}$, attach membership certificates to the enumeration of A and retain just entries satisfying p. Classical decidability of p allows this filtering. Every subset element is retained because its certificate proves the test true. The resulting definition is noncomputable for an arbitrary proposition p; no algorithm deciding p is claimed.

Lean: `Enumeration.subtype`

For a dependent sum $\sum_{a \in A} B_a$, enumerate each fibre, tag it with a, and concatenate over the enumeration of A. A pair (a, b) appears because a occurs in the outer list and b in the enumeration of its fibre.

Lean: `Enumeration.sigma`

Definition 11.2 (Bijection)**.** A bijection $f : A \cong B$ consists of functions $f : A \to B$ and $f^{-1} : B \to A$ with both inverse equations, pointwise.

Lean: `Bijection`

Identity uses the identity function twice. Inversion interchanges the two functions and their inverse laws. Composition uses $g \circ f$ forwards and $f^{-1} \circ g^{-1}$ backwards; cancelling the inner inverse pair and then the outer pair proves each law.

Lean: `Bijection.refl`

Lean: `Bijection.symm`

Lean: `Bijection.trans`

Bijections extend to disjoint unions by applying the appropriate map inside each tag. Case analysis on the tag reduces both inverse laws to the component laws.

Lean: `Bijection.sum`

Every bijection is injective: apply its inverse to an equality of images and cancel. An enumeration transports along a bijection by mapping its forward function over the list. For $b \in B$, the listed preimage $f^{-1}(b)$ maps to b, proving completeness.

Lean: `Bijection.injective`

Lean: `Bijection.enumeration`

Lemma 11.3 (Distinctness under restricted injectivity)**.** *If a list l has no repeated elements and f is injective on elements of l, then* $\mathrm{map}(f, l)$ *has no repeated elements.*

Proof. Distinct positions in l contain distinct elements. If their images were equal, restricted injectivity would identify those elements, a contradiction. Lean expresses this as preservation of the pairwise inequality predicate under mapping. □

Lean: `nodup_map_on`

This restricted version will be essential: replacement can identify an input with an output, while remaining injective on each incidence list separately.

11.2 Ports, signatures, and graph invariants

Definition 11.4 (Signature and ports)**.** A signature σ assigns an ordered input-label list and output-label list to every operation. With internal carrier V and boundary lengths n, m, vertices are the disjoint union

$$\mathrm{Port}(V, n, m) = V + \mathrm{Fin}(n) + \mathrm{Fin}(m).$$

The three constructors are internal, input, and output. In particular the two boundary sets are disjoint even if their wire labels agree.

Lean: `Signature`

Lean: `Port`

The predicates IsInput and IsOutput test the corresponding constructor. A carrier map $f : V \to W$ acts only on internal vertices and fixes each boundary position. It is injective whenever f is: unlike constructors cannot have equal images, and equality within the internal constructor reduces to injectivity of f. The input and output predicates are preserved and reflected by this map, by the same three-case analysis.

Lean: `Port.IsInput`

Lean: `Port.IsOutput`

Lean: `Port.map`

Lean: `Port.map_injective`

Lean: `Port.map_isInput`

Lean: `Port.map_isOutput`

Given boundary-label lists I, O and internal labelling $\lambda : V \to L$, the label of an internal

vertex is $\lambda(v)$, of input i is I_i, and of output j is O_j. The finite-index types ensure that these lookups are defined.

Lean: `Port.label`

Definition 11.5 (Concrete typed net)**.** A net $G : I \to O$ has finite internal and edge carriers, labels for both, and ordered source and target vertex lists $s(e), t(e)$ for each edge. It satisfies:

1. Each individual source list and target list has no repetitions.
2. A vertex belongs to the sources of at most one edge, and to the targets of at most one edge.
3. A vertex belongs to some source list exactly when it is not an output; it belongs to some target list exactly when it is not an input.
4. Mapping vertex labels over $s(e)$ and $t(e)$ gives precisely the input and output lists of the signature of the edge label.

Lean: `Net`

These fields are part of a net's certificate of well-formedness. Coverage excludes isolated vertices. It does not exclude cycles or a vertex belonging to both the source and target of the same edge. The full vertex type and its label function are abbreviations for the port type and the labelling just defined.

Lean: `Net.Vertex`

Lean: `Net.vertexLabel`

From forward source coverage, membership in $s(e)$ implies that a vertex is not an output. From target coverage, membership in $t(e)$ implies that it is not an input. Apply the appropriate equivalence to the witness edge e.

Lean: `Net.source_not_output`

Lean: `Net.target_not_input`

Taking lengths in the two typing equations shows that incidence-list lengths are exactly the corresponding signature arities: mapping labels does not change length.

Lean: `Net.source_length`

Lean: `Net.target_length`

The ith source port of e is its source list indexed by a position in the signature's input list; the preceding length equality justifies the lookup. Target ports are defined in the same way with the output list.

Lean: `Net.sourcePort`

Lean: `Net.targetPort`

Each indexed port belongs to its incidence list, by the list lookup membership lemma. Within either list, equality of indexed ports implies equality of indices, because that list has no duplicates; equality of finite ordinals follows from equality of their underlying natural numbers.

Lean: `Net.sourcePort_mem`

Lean: `Net.targetPort_mem`

Lean: `Net.sourcePort_injective`

Lean: `Net.targetPort_injective`

The label of source port i is the ith signature input label, and the label of target port i is the ith signature output label. Apply indexed lookup to the appropriate list typing equation, then commute lookup with mapping.

Lean: `Net.sourcePort_label`

Lean: `Net.targetPort_label`

Finally, membership in a source list is equivalent to being its indexed port for some signature input position. Given membership, choose its list index and transport the bound along the arity equality. Conversely, indexed ports belong to their lists. The target statement uses the output arity and the identical membership argument.

Lean: `Net.mem_source_iff_port`

Lean: `Net.mem_target_iff_port`

11.3 Elementary graph constructors

The ordered finite range has no duplicates: equality of its entries at two positions gives equality of their underlying natural numbers and hence of the positions.

Lean: `finRange_nodup`

Definition 11.6 (Corolla and empty graph)**.** The corolla of operation o has one edge labelled o, no internal vertices, and all of its prescribed inputs and outputs in signature order. Its sources are precisely the input constructors and its targets the output constructors.

Well-formedness. Finite ranges and their constructor maps give distinct incidence lists. There is only one edge, so edge uniqueness is immediate. Every input occurs in its source range and every output in its target range, and the opposite boundary constructors cannot occur there. There are no internal vertices to check. Mapping labels over either ordered range recovers the corresponding signature list, proving typing. □

Lean: `Net.corolla`

The empty graph has empty internal and edge carriers and empty boundaries. Each incidence or label function has empty domain; all obligations follow by eliminating the purported element of an empty type. A bare identity wire is not provided: a single vertex serving as both boundary input and boundary output would violate the disjoint-boundary representation used here.

Lean: `Net.empty`

12 Shallow plugging, including self-loop holes

Fix a host $C : I \to O$, an edge h to remove, and a replacement H whose boundary lists are exactly the input and output signature lists of C's label at h. All constructions in this section are in the namespace Net.Replacement unless explicitly stated otherwise.

12.1 Carriers and boundary identification

Definition 12.1 (Replacement carriers)**.** The new internal vertices are $C_V + H_V$, with disjoint tags providing freshness. The new edges are $(C_E \setminus \{h\}) + H_E$. The outer boundary remains I, O. The surviving-edge subtype carries a proof of inequality to h, so the removed edge cannot accidentally survive as an old edge.

Lean: `Net.Replacement.Vertex`

Lean: `Net.Replacement.Edge`

The host embedding j tags each old internal vertex on the left and fixes the outer boundary. The insertion map k tags a replacement internal vertex on the right, sends replacement input i to $j(s_h(i))$, and sends replacement output i to $j(t_h(i))$. Thus gluing is expressed by a function rather than a quotient of all vertex names.

Lean: `Net.Replacement.host`

Lean: `Net.Replacement.insert`

The host embedding is injective by injectivity of the left sum tag and the port map lemma. Its image is disjoint from freshly tagged replacement internals: case analysis on a host vertex gives either a boundary constructor or an internal left tag, neither equal to an internal right tag.

Lean: `Net.Replacement.host_injective`

Lean: `Net.Replacement.host_ne_internal`

Lemma 12.2 (The two restricted insertion injections)**.** *The insertion map is injective on vertices that are not outputs. It is also injective on vertices that are not inputs.*

Proof. For the first claim, exclude output cases using the hypotheses. Two internal images agree only when their right-tagged names agree. An internal image cannot agree with an input image, by freshness. Two input images agree only when their host source ports agree, and source-port injectivity identifies their indices. For the second claim, exclude input cases instead, use target-port injectivity on two outputs, and use the same freshness argument on mixed cases. □

Lean: `Net.Replacement.insert_source_injective`

Lean: `Net.Replacement.insert_target_injective`

If a non-output replacement vertex has insertion image $j(v)$, it must be some input i, with $s_h(i) = v$. Outputs are excluded by hypothesis, internals by freshness, and in the input case host injectivity gives the equation. Dually, a non-input replacement vertex mapping to $j(v)$

must be output i with $t_h(i) = v$. These inversion lemmas control mixed old/new incidence cases.

Lean: `Net.Replacement.insert_source_host`

Lean: `Net.Replacement.insert_target_host`

12.2 Preservation of incidence invariants

Sources and targets of surviving host edges are their old lists mapped by j. Those of inserted edges are the replacement lists mapped by k.

Lean: `Net.Replacement.source`

Lean: `Net.Replacement.target`

Lemma 12.3 (Distinct incidence entries)**.** *Every new source list and every new target list has no duplicates.*

Proof. A surviving host list uses the injective map j. A replacement source list contains no outputs, so the first restricted injection of k applies to its members. A replacement target list contains no inputs, so the second injection applies. In all cases use preservation of distinctness under restricted injectivity. □

Lean: `Net.Replacement.source_nodup`

Lean: `Net.Replacement.target_nodup`

Lemma 12.4 (Unique source and target incidence)**.** *A vertex belongs to the sources of at most one new edge, and to the targets of at most one new edge.*

Proof. For source incidence, split on the two edge tags. Two host edges reduce, by injectivity of j, to old source uniqueness. Two inserted edges reduce, by restricted source injectivity of k, to source uniqueness in H. In the mixed case the shared vertex is an inserted input, by the source inversion lemma; its corresponding host vertex is therefore a source port of h. Old source uniqueness identifies the surviving host edge with h, contradicting its subtype certificate. The other mixed ordering is the same contradiction with the equality reversed. For target incidence, repeat the four cases with target uniqueness, restricted target injectivity, and the target inversion lemma: the mixed vertex is now an inserted output and a target port of h. □

Lean: `Net.Replacement.source_unique`

Lean: `Net.Replacement.target_unique`

A vertex in a new source list is not an outer output. In the host case use the old source property and preservation of boundary constructors. In the inserted case an internal vertex remains internal, an input maps to a source port of h, and an output is impossible in a replacement source list. For a new target list, exchange source/input with target/output: inserted inputs are impossible and inserted outputs map to host target ports, which cannot be outer inputs.

Lean: `Net.Replacement.source_not_output`

Lean: `Net.Replacement.target_not_input`

Lemma 12.5 (Coverage of surviving host vertices)**.** *For every host vertex v that is not an output, $j(v)$ belongs to a new source list. If v is not an input, $j(v)$ belongs to a new target list.*

Proof. Choose an old source edge by coverage in C. If it differs from h, retain it. If it equals h, express v as source port i; replacement input i belongs to a source edge by coverage in H, and its image is $j(v)$. For target coverage, choose an old target edge. Retain it unless it is h; in that case express v as target port i and use coverage at replacement output i. □

Lean: `Net.Replacement.source_host_covered`

Lean: `Net.Replacement.target_host_covered`

Full source coverage now follows by splitting a new vertex into a host internal, a fresh internal, an outer input, or an outer output. Host internals and inputs use the preceding lemma. Fresh internals use replacement coverage and map the witness edge into the right edge tag. Outer outputs contradict the required non-output hypothesis. The reverse implication is the source exclusion already proved. Full target coverage uses host internals and outer outputs, replacement coverage for fresh internals, and excludes outer inputs; its reverse implication is target exclusion.

Lean: `Net.Replacement.source_coverage`

Lean: `Net.Replacement.target_coverage`

12.3 Labels, typing, and the certified result

New internal labels use the host label on left tags and replacement label on right tags. Surviving edges keep their labels; inserted edges receive their replacement labels.

Lean: `Net.Replacement.internalLabel`

Lean: `Net.Replacement.edgeLabel`

Labelling commutes with j, by the three port cases. It also commutes with k: internal labels agree by definition, while input and output labels agree by the host indexed-port typing lemmas and the matching replacement interface.

Lean: `Net.Replacement.host_label`

Lean: `Net.Replacement.insert_label`

For source typing, split the edge tag and compose the incidence map with the vertex-label map. The commuting equation just proved reduces the result to source typing in C or H, respectively. For target typing the same composition argument reduces to their target typing equations.

Lean: `Net.Replacement.source_typed`

Lean: `Net.Replacement.target_typed`

Theorem 12.6 (Well-formed shallow plugging)**.** *There is a net $C[h := H] : I \to O$ with the carriers, labels, and incidence lists above. It preserves the entire ordered external interface and all graph invariants, even when h has a vertex in both its source and target lists.*

Proof. Enumerate the new internal carrier by the sum enumeration. Enumerate surviving host edges by subset filtering, and concatenate the tagged replacement-edge enumeration. Supply each of the distinctness, uniqueness, coverage, and typing proofs established above as the corresponding net field. This assembles the certified graph. Classical filtering of the surviving-edge subtype accounts for the noncomputable qualifier. □

Lean: `Net.plug`

There is deliberately no claim that k is globally injective in this theorem. If a host source port equals a host target port, its replacement input and output are identified. The separate source and target injection arguments are sufficient for every net invariant.

13 Paths, renaming, and isomorphism

13.1 Paths and their behaviour under plugging

Definition 13.1 (Adjacency and path)**.** Edges e, f are adjacent when some vertex belongs to both $t(e)$ and $s(f)$. A path is a nonempty edge list: a singleton is a path; prepending e to a path starting at f gives a path when e, f are adjacent. Edges may repeat, and this notion does not specify a chosen boundary vertex at either end.

Lean: `Net.Adjacent`

Lean: `Net.Path`

For a list beginning e, f, the path predicate is equivalent to adjacency of e, f together with the path predicate for the tail starting at f. Invert the path constructor for the forward implication and apply it for the reverse one.

Lean: `Net.path_pair_iff`

Any map on edges preserving adjacency maps paths to paths. Induct on the path: singletons remain singletons, and the inductive step transports its first adjacency and then its tail path. If the edge map preserves and reflects adjacency, it preserves and reflects paths. Induct on the list: the empty list is a path on neither side, singletons are paths on both sides, and longer lists reduce to the first adjacency equivalence and the tail induction hypothesis.

Lean: `Net.Path.map`

Lean: `Net.path_map_iff`

Lemma 13.2 (Host and inserted adjacencies)**.** *Two surviving host edges are adjacent after plugging exactly when they were adjacent before plugging. Two inserted edges that were adjacent remain adjacent.*

Proof. For host reflection, pull both incidence witnesses back through their mapped lists. Equality of their host images implies equality of the original vertices by injectivity of j, giving an old shared witness. For host preservation map an old shared witness through j. For inserted preservation map the shared replacement witness through k; injectivity is unnecessary in this direction. □

Lean: `Net.Replacement.adjacent_host_iff`

Lean: `Net.Replacement.adjacent_insert`

Definition 13.3 (Non-self-loop hole)**.** The hole h is non-self-looping if no vertex belongs to both $s(h)$ and $t(h)$. This requires the two lists to be disjoint; it is stronger than distinctness within each individual list, already required of every net.

Lean: `Net.Replacement.NonSelfLoop`

For such a hole, k is globally injective. Equal internal images reduce to equal names; mixed internal/boundary cases contradict freshness. Two input images or two output images reduce to the appropriate port injection. Equality of an input image and an output image would exhibit a host vertex in both $s(h)$ and $t(h)$, contrary to non-self-looping.

Lean: `Net.Replacement.insert_injective`

Consequently inserted adjacency is reflected as well as preserved: pull the two mapped-list witnesses back and use global injectivity of k to identify them. The other implication is the unconditional preservation result above.

Lean: `Net.Replacement.adjacent_insert_iff`

Theorem 13.4 (Path preservation and reflection)**.** *A list of surviving host edges is a path after plugging if and only if its old edge list is a path. Any replacement path gives an inserted path. For a non-self-loop hole the converse holds for inserted paths too.*

Proof. For host paths, induct on the surviving-edge list. Empty and singleton lists are immediate; a longer list reduces to host adjacency equivalence and the tail hypothesis. For inserted preservation, apply path mapping to inserted adjacency preservation. For inserted reflection under the stated restriction, apply the general path equivalence lemma to inserted adjacency equivalence. □

Lean: `Net.Replacement.path_host_iff`

Lean: `Net.Replacement.path_insert`

Lean: `Net.Replacement.path_insert_iff`

13.2 Transport along carrier bijections

A bijection on internal names extends to ports by mapping only internal vertices. Its inverse laws follow by cases on a port: internal cases use the original inverse laws and boundary cases are reflexive. Mapping the identity is pointwise the identity, also by port cases; function extensionality gives the corresponding identity of functions.

Lean: `Port.bijection`

Lean: `Port.map_id`

Lean: `Port.map_id_fun`

Fix bijections $a : G_V \cong V'$ and $b : G_E \cong E'$. Reindexed sources and targets at $e' \in E'$ are the old incidence lists at $b^{-1}(e')$, mapped by the port extension of a.

Lean: `Net.Reindex.source`

Lean: `Net.Reindex.target`

Source uniqueness survives this transport: two shared mapped witnesses are equal by port injectivity, so old source uniqueness identifies their old edge indices; apply b and cancel $b \circ b^{-1}$. The target uniqueness proof uses old target uniqueness and the same cancellation.

Lean: `Net.Reindex.source_unique`

Lean: `Net.Reindex.target_unique`

For source coverage, a new source witness pulls back to an old source vertex, which is not an output; port maps preserve this property. Conversely, pull a non-output vertex backwards through a^{-1}, choose an old source edge by coverage, and map both the edge and vertex forwards. The two inverse laws prove that this is a witness for the original new vertex. Target coverage repeats this argument with non-input vertices and old target coverage.

Lean: `Net.Reindex.source_coverage`

Lean: `Net.Reindex.target_coverage`

Give an internal name w the label of $a^{-1}(w)$. Labelling an old vertex after mapping by a then agrees with its old label: the internal case cancels $a^{-1} \circ a$, and boundary labels are fixed.

Lean: `Net.Reindex.label_map`

Definition 13.5 (Reindexed graph)**.** The reindexed net uses the new carriers, inverse-transported labels, and the incidence lists just defined.

Well-formedness. Transport finite enumerations along the bijections. Distinctness follows from port injectivity, uniqueness and coverage from the preceding lemmas. For typing, compose incidence mapping with the vertex-label map, use label commutation, and apply the original typing equation. The boundary lists are unchanged. □

Lean: `Net.reindex`

13.3 Graph isomorphism and shallow context congruence

Definition 13.6 (Isomorphism)**.** An isomorphism $G \cong H$ between nets with the same signature and boundary lists is a bijection of internal vertices and a bijection of edges preserving internal labels, edge labels, and both ordered incidence lists. Its port action fixes boundary positions.

Lean: `Net.Iso`

Identity bijections and identity list maps yield a reflexive isomorphism. For the inverse isomorphism, invert both bijections. Inverse label preservation follows by applying the original equation at an inverse image and cancelling. To invert an incidence equation, map its two sides by the inverse port map and cancel the forward/inverse maps pointwise. For composition, compose carrier bijections; compose label equalities, and use successive list mapping followed by the two incidence-preservation equations.

Lean: `Net.Iso.refl`

Lean: `Net.Iso.symm`

Lean: `Net.Iso.trans`

Isomorphisms preserve and reflect adjacency: transport a shared witness forwards for preservation; for reflection pull both witnesses back from the incidence maps and identify their preimages by port injectivity. Applying the general path equivalence lemma then preserves and reflects paths, with their edge lists mapped by the edge bijection.

Lean: `Net.Iso.adjacent_iff`

Lean: `Net.Iso.path_iff`

The original graph is isomorphic to its reindexing. Use the chosen carrier bijections; inverse cancellation proves label preservation and each incidence identity directly from the definition of reindexing.

Lean: `Net.reindexIso`

Suppose $f : H \cong K$ and both graphs have the interface of host hole h. The intended isomorphism between $C[h := H]$ and $C[h := K]$ is the identity on host internal names and f on replacement internal names, and likewise the identity on surviving host edges and f on inserted edges. Sum bijections supply their inverse laws.

Lean: `Net.PlugIso.vertices`

Lean: `Net.PlugIso.edges`

The resulting port map commutes with host embedding: check the three port constructors, all of which are reflexive. It commutes with insertion too: internal vertices use the replacement part of the sum map, and boundary vertices reduce to the host-embedding equation at the corresponding hole port.

Lean: `Net.PlugIso.map_host`

Lean: `Net.PlugIso.map_insert`

Theorem 13.7 (Congruence of shallow plugging)**.** *For every typed host and hole, $H \cong K$ implies $C[h := H] \cong C[h := K]$. No non-self-loop hypothesis is required.*

Proof. Use the sum bijections above. For each label equation split the carrier tag: host labels are unchanged and inserted labels use f's corresponding equation. For a host source list, commute the new port map past host embedding pointwise. For an inserted source list, commute it past insertion and then use source preservation by f. Repeat with target lists and target preservation. These arguments prove all isomorphism fields without reflecting through insertion. □

Lean: `Net.plugIso`

Here edge labels are preserved by literal equality. For boxed labels containing bodies, this is not yet the recursive notion that would identify independently renamed but isomorphic bodies. No such stronger congruence is claimed.

14 Concrete uniqueness of operation paths

Choose a predicate P on operation labels in a net G. This section first proves a generic graph theorem; the next section shows that the single-source condition is automatic for the typed operation labels of the linear hierarchy.

Definition 14.1 (Unary operations and traversal)**.** Selected operations are unary when each edge whose label satisfies P has a source list of length one. An operation step $s \rightsquigarrow t$ chooses such an edge e with $s \in s(e)$ and $t \in t(e)$.

Lean: `Net.UnaryOperations`

Lean: `Net.OperationStep`

Reverse traversal is the machine whose transition from t to s is $s \rightsquigarrow t$ and whose successful final states are boundary inputs. Inputs are normal: any reverse step from an input would put that input in a target list, contradicting target exclusion.

Lean: `Net.reverseOperations`

Lemma 14.2 (Determinism derived from incidence)**.** *If selected operations are unary, reverse operation traversal is deterministic.*

Proof. Two reverse steps from the same vertex select edges both targeting that vertex. Target uniqueness identifies the edges. Their source list has length one, so both predecessor vertices equal its only entry and hence equal one another. There is no additional determinism premise. □

Lean: `Net.reverseOperations_deterministic`

Definition 14.3 (Edge-recording trace)**.** A reverse operation trace from v to w records its actual edge list. The empty trace starts and ends at the same vertex. A nonempty trace first chooses an operation edge targeting the current vertex and a vertex in its source, then continues recursively from that source vertex.

Lean: `Net.OperationTrace`

A trace with edge list l yields exactly $|l|$ steps in the reverse machine. Induct on its construction: the empty trace is a zero-step execution and each trace constructor supplies the first step of the counted execution.

Lean: `Net.OperationTrace.steps`

Theorem 14.4 (Exact uniqueness to inputs)**.** *Under the unary condition, two reverse traces from the same vertex to inputs have equal edge lists and equal ending input vertices.*

Proof. Induct on the first trace. If it is empty, the starting vertex is an input; the second trace cannot be nonempty, since no input belongs to a target list. Thus it too is empty and both conclusions follow. If the first trace is nonempty, the second cannot be empty, for the same target-exclusion reason. Their first edges are equal by target uniqueness. Their next vertices agree by the determinism lemma. Apply the induction hypothesis to their tails and prefix the common edge to the resulting list equality. □

Lean: `Net.OperationTrace.unique_to_inputs`

A forward operation path from s to t with edge list l means a reverse trace from t to s with list reverse(l). Empty paths are allowed in this endpoint-sensitive notion, unlike the nonempty edge-only path predicate of the preceding section.

Lean: `Net.OperationPath`

Two forward operation paths from inputs to the same vertex therefore have equal forward edge lists and equal starting input indices. Apply exact trace uniqueness, use injectivity of list reversal, and then injectivity of the input constructor.

Lean: `Net.operationPath_unique_from_inputs`

Theorem 14.5 (No positive cycle on an input-rooted operation path)**.** *If a vertex v is reached by a forward operation path from an input, every reverse operation trace from v back to v has an empty edge list.*

Proof. The input-rooted path gives a reverse execution from v to a successful input. Prefix that execution by the proposed cycle to obtain another successful execution from v to the same input. Deterministic terminal uniqueness identifies their lengths. Thus $|cycle| + |path| = |path|$, so the cycle has length zero and is empty. □

Lean: `Net.no_operation_cycle_on_input_path`

This is a statement about cycles at input-reachable vertices, not a global acyclicity restriction on nets. The self-loop example below is a valid net with a cycle and no boundary inputs.

15 Typed boxes, deep occurrences, and contexts

15.1 Relabelling without changing incidence

A signature-preserving map f of operation alphabets relabels a net by replacing each edge label o with $f(o)$. All carriers, enumerations, vertex labels, and incidence data are retained. Hence their invariant proofs are retained too. For typing, compose the old typing equality with the inverse of the stipulated signature equality for $f(o)$. This yields the new signature list at that edge.

Lean: `Net.relabel`

15.2 The linear wire and label discipline

Definition 15.1 (Wire types, operation arities, and holes)**.** A wire type is either a term wire $*$ or a thunk wire $T^n(*)$, carrying a natural number n of bound variables. An operation specification assigns each operation an argument count $a(o)$ and an ordered list of binder counts $b(o)$. A hole label contains a natural-number name together with arbitrary input and output wire lists. Equality of hole labels includes these lists as well as the name.

Lean: `Linear.Wire`

Lean: `Linear.Operations`

Lean: `Linear.Hole`

Primitive edge labels are operations, variables, focus labels from the abstract three-mode type, or typed holes. This introduces concrete focus *labels*; it does not yet define the UAM transition relation.

Lean: `Linear.Primitive`

Their signature is as follows. Every non-hole primitive has one term input. Operation o has $a(o)$ term outputs followed by the thunk outputs specified in $b(o)$. A variable or focus has one term output. A hole uses precisely the input and output lists stored in its label. Ordering is part of each of these definitions.

Lean: `Linear.primitiveSignature`

Definition 15.2 (Alphabets and typed boxes)**.** An alphabet is a label type and a signature on it. A box over alphabet A stores numbers n, m and a body net over A of type

$$* \longrightarrow *^{n+m}.$$

The corresponding outer box edge will have type $T^n(*) \longrightarrow *^m$. Thus body boundary and outer boundary differ: n body outputs account for binding, and m for exposed free variables.

Lean: `Linear.Alphabet`

Lean: `Linear.Box`

All alphabet label types live in one fixed higher universe, while graph carriers remain small types. The base alphabet contains just primitives, raised into that universe by a universe lift; its signature reads the underlying primitive. The successor alphabet is the disjoint union of

primitives and boxes over the previous alphabet. Its signature uses the primitive signature on the left and the stated external box interface on the right.

Lean: `Linear.baseAlphabet`

Lean: `Linear.nextAlphabet`

Recursively, depth zero uses the base alphabet and depth $d + 1$ uses the successor of the depth-d alphabet. A hypernet at depth bound d is a net over that alphabet. The bound limits possible nesting; it need not be the actual minimum depth of the particular graph.

Lean: `Linear.alphabet`

Lean: `Linear.Hypernet`

The operation predicate selects primitive operation labels only. At depth zero it inspects the universe-lifted primitive; at a successor depth it accepts only the primitive-operation summand, excluding boxes and all other primitive kinds.

Lean: `Linear.IsOperation`

Selected operation edges have exactly one source. Rewrite the incidence length using signature typing, split on the depth and label constructors, discard labels excluded by the predicate, and compute the operation input list $[*]$.

Lean: `Linear.unary_operations`

It follows that two operation paths from inputs to the same vertex have identical edge lists and input indices in every typed hypernet. Instantiate the generic operation-path uniqueness theorem with the preceding derived unary proof. This is the graph-theoretic uniqueness conclusion associated with the paper's Lemma 22 for this representation; neither determinism nor a locality field is assumed.

Lean: `Linear.operation_path_unique`

15.3 Box construction and promotion of the depth bound

The box constructor takes a body with the specified $n + m$ output interface and forms the corolla of its box label in the successor alphabet. Corolla well-formedness proves all outer graph conditions. The result has exactly one outer box edge and the external type $T^n(*) \to *^m$.

Lean: `Linear.box`

Promotion raises the available depth bound by one without changing the outer graph. At depth zero, send a lifted primitive to the primitive summand at depth one. At a successor depth, keep primitives and recursively promote each box body, retaining its bound/free metadata. These maps preserve signatures because neither primitive signatures nor external box arities change. Apply the generic relabel construction at each recursive step.

Lean: `Linear.raise`

Outer sources and targets are unchanged by promotion. The code records these as heterogeneous equalities: the dependent graph types are indexed by different depths, although after unfolding either defining case the incidence functions are definitionally the same. Splitting on depth proves both equalities by reflexivity.

Lean: `Linear.raise_source`

Lean: `Linear.raise_target`

15.4 Finite occurrence addresses at arbitrary depth

Definition 15.3 (All edge occurrences)**.** At depth zero, an occurrence is a shallow edge. At depth $d + 1$, an occurrence is either a shallow edge or a pair (e, a) where e is a box edge and a an occurrence in its body. A primitive edge has the empty type of child occurrences.

Lean: `Linear.AllEdge`

The parent edge is part of every deep address. In particular identical bodies placed at different parent edges do not identify their child occurrences.

Theorem 15.4 (Finiteness of all occurrences)**.** *Every finite-depth hypernet has an explicit finite enumeration of its shallow and deep edge occurrences.*

Proof. Recurse on the depth bound. At zero use the graph's edge enumeration. At a successor bound, enumerate shallow edges in the left summand. For the right summand enumerate the dependent sum over outer edges: a primitive contributes an empty enumeration, and a box contributes the recursively obtained enumeration of its body. The sum and dependent-sum enumeration constructions already prove completeness of this list. Recursion decreases the depth at every box descent. □

Lean: `Linear.allEdgesFinite`

Every given occurrence belongs to this enumeration, by its completeness field. This is the finiteness content of Appendix A, Lemma 19, for the typed hierarchy. The list need not be duplicate-free, since completeness suffices here.

Lean: `Linear.every_edge_enumerated`

15.5 Hole lookup and the context condition

A primitive hole lookup returns its stored label for a hole constructor and no label otherwise. Full occurrence lookup uses that function on a shallow primitive, returns no hole at a shallow box edge, and recursively looks inside a deep box address. A purported child occurrence under a primitive has empty type and is eliminated. Dependent transports in Lean express that the selected label really is the box whose body is being searched.

Lean: `Linear.primitiveHole`

Lean: `Linear.holeAt`

Definition 15.5 (Unique holes and contexts)**.** A hypernet has unique holes if, for each complete typed hole label h, any two occurrences whose lookup returns h are equal. A context is a hypernet together with this property. This permits multiple different hole labels and also permits no holes; it is an at-most-once condition for each label over all depths.

Lean: `Linear.UniqueHoles`

Lean: `Linear.Context`

Lemma 15.6 (Boxing preserves unique holes)**.** *If a body has unique holes, its one-edge box has unique holes.*

Proof. A shallow occurrence in the outer box is not a hole, so either proposed hole occurrence must be deep. The outer graph has only one edge; therefore their parent indices agree. Their lookups are the body lookups, whose occurrences agree by the body's uniqueness assumption. Tagging these equal child addresses with their common parent gives equality of the original occurrences. □

Lean: `Linear.box_unique_holes`

Boxing a context now means boxing its underlying graph and supplying this lemma as the context certificate. Shallow plugging at any fixed hierarchy depth is just the generic typed net plugging construction with that depth's alphabet; its interface constraints are checked by the type of its replacement argument.

Lean: `Linear.boxContext`

Lean: `Linear.plug`

This last operation returns a hypernet. It is not yet a general theorem that plugging two contexts yields a context: freshness of hole labels and its preservation proof remain required. Likewise the typed hierarchy has not yet been proved adequate for the paper's original set-based presentation of graph-labelled edges.

16 Addressed deep replacement and concrete locality

16.1 Changing one box body

A per-edge signature-preserving relabelling generalizes alphabet relabelling: it assigns a new label directly to each edge, rather than requiring the assignment to factor through its old label. All carriers, incidence, and vertex labels stay unchanged. The two new typing equations follow by composing each old typing proof with the inverse of the specified per-edge signature equality.

Lean: `Net.relabelEdges`

Definition 16.1 (Replacement of a box label)**.** Suppose outer edge e is labelled by a box (n, m, B). Given a new body B' of the same body type $* \to *^{n+m}$ and depth bound, change just the label at e to (n, m, B'), retaining every other edge label.

Well-formedness. Use classical equality testing on outer edges to select e. At e the old and new labels have the same external input $[T^n(*)]$ and output $*^m$; this follows from the given equality identifying the old label as the box. At every other edge the label is unchanged. These two cases prove the per-edge signature conditions for relabelling, which supplies the certified net. □

Lean: `Linear.replaceBox`

Selection uses an edge occurrence, not equality of box contents. Equal bodies at other edges therefore do not cause simultaneous replacement.

16.2 Typed sites and replacement at a site

Definition 16.2 (Site)**.** A site in an outer hypernet G carries a local depth bound and two wire-label lists J, K. It is either a shallow edge of G, in which case J, K are exactly that edge's signature lists, or a box edge with a proof identifying its box label together with a recursively chosen site in its body. In the latter case the site's local depth and interface are inherited from the child site.

Lean: `Linear.Site`

The indices prevent supplying a replacement of the wrong interface or local depth. A site may select an arbitrary edge, not only an edge labelled as a hole; the separate unique-hole context condition is not assumed here.

Definition 16.3 (Deep plugging)**.** To replace at a shallow site use shallow net plugging. To replace at a site inside a box, recursively replace in its body and use box-label replacement to put that modified body back at the chosen outer edge. The recursion is on the site's finite constructor structure, and the result retains the outer depth bound and boundary interface.

Lean: `Linear.plugAt`

At a shallow site this definition agrees with shallow plugging by reflexivity: both sides unfold to the same term.

Lean: `Linear.plugAt_shallow`

Theorem 16.4 (Locality at every enclosing box)**.** *Replacement strictly inside a selected box leaves all outer source lists and all outer target lists unchanged. It also leaves the label of every other outer edge unchanged.*

Proof. The inside branch changes only an edge label using per-edge relabelling. The source and target fields of that construction are definitionally those of the old graph, so their equalities are reflexive, independent of what the recursive replacement did inside. For another outer edge $f \neq e$, the conditional defining box-label replacement chooses the old-label branch, proving the label equation. Repeated descent applies this fact at each enclosing layer. □

Lean: `Linear.plugAt_inside_source`

Lean: `Linear.plugAt_inside_target`

Lean: `Linear.plugAt_inside_other_label`

These are unconditional, concrete graph locality results about the defined replacement operation. They do not yet prove UAM traversal locality, refocusing, or the specimen interface laws: no concrete machine transition relation has been constructed. Nor do they yet establish associativity of context plugging or a recursive isomorphism theorem for deep replacement.

17 Concrete graph examples and regression proofs

17.1 Self-loop gluing and the necessary reflection restriction

Use a singleton wire-label set and a singleton operation alphabet, whose only operation has one input and one output of that label. The graph named loop has one internal vertex, one edge, no outer boundary, and that vertex as both the edge's source and target.

Lean: `Examples.unary`

Lean: `Examples.loop`

Its finite carriers are both Fin(1). Singleton incidence lists are distinct; all edges are equal since their indices are bounded by one. The only internal vertex is incident on both sides, providing coverage, while boundary cases have empty finite-index types. Label typing is reflexive. Thus this is a well-formed net despite its cycle. Let wireEdge be the corolla of the same unary operation, with its two distinct boundary vertices.

Lean: `Examples.wireEdge`

The corolla is not adjacent to itself: a shared target/source witness would have to be both its output constructor and its input constructor, an impossible constructor equality.

Lean: `Examples.corolla_not_self_adjacent`

When the corolla replaces the loop's sole edge, its input and output images are equal. Both are, by definition, the same old internal vertex, because the host hole's source and target port coincide. Hence the inserted edge is adjacent to itself: use that old vertex as the witness and compute the mapped singleton source and target lists.

Lean: `Examples.inserted_boundary_identified`

Lean: `Examples.inserted_self_adjacent`

Consequently the inserted two-entry list $[e, e]$ is a path, while $[e, e]$ is not a path in the original corolla. The positive part uses its new self-adjacency and a singleton tail path. The

negative part extracts its first adjacency and contradicts the corolla result. This proves that unconditional reflection of inserted paths would be false; the non-self-loop hypothesis cannot simply be omitted from that theorem.

Lean: `Examples.unconditional_path_reflection_fails`

The original loop itself has the two-entry path $[e, e]$, using its sole internal vertex as the shared witness. This verifies that the graph representation has not silently imposed global acyclicity.

Lean: `Examples.loop_is_cyclic`

The finite ordinal with one element is bijective to the unit type: send its index to the sole unit element and send unit back to zero. The inverse laws follow respectively from the ordinal bound and unit case analysis. An anonymous example reindexes the corolla's edge carrier along this bijection and applies shallow plugging congruence. It obtains isomorphic filled loops despite the different replacement-edge carrier types, testing actual renaming rather than reflexivity alone.

Lean: `Examples.oneToUnit`

Lean: `Examples.example (line 89)`

17.2 Values, boxes, and distinguished occurrence addresses

For the linear examples, take two operation labels, value and other-value. Give both zero term arguments and no thunk arguments; each therefore has signature $* \to []$. The graph value is the corolla of the first operation at depth zero.

Lean: `Linear.Examples.Op`

Lean: `Linear.Examples.operations`

Lean: `Linear.Examples.value`

Boxing it with zero bound and free variables yields a closed box of type $T^0(*) \to []$ at depth one. A further example constructs the analogous graph at depth bound two by putting a primitive value at depth one inside a box. This is a larger *available bound*, not a claim that the example has two actually nested boxes.

Lean: `Linear.Examples.boxedValue`

Lean: `Linear.Examples.higherBoxedValue`

An anonymous proof verifies that any occurrence in the depth-one boxed value belongs to the recursive occurrence enumeration, by the general completeness theorem. The outer box edge is represented by a shallow left-tagged occurrence; the value edge in its body is a right-tagged pair of parent edge and child edge.

Lean: `Linear.Examples.example (line 117)`

Lean: `Linear.Examples.outerOccurrence`

Lean: `Linear.Examples.innerOccurrence`

These two addresses are unequal. If equal, applying the sum-tag test would give true = false, which is impossible. This checks separation of outer and inner occurrences even though each

individual edge carrier is singleton.

Lean: `Linear.Examples.outer_ne_inner`

The other-value corolla has exactly the same interface as the value corolla. A site inside the boxed value descends through its only box edge and selects the value edge in its body. Replacing at this site with other-value defines replacedInside.

Lean: `Linear.Examples.otherValue`

Lean: `Linear.Examples.insideValue`

Lean: `Linear.Examples.replacedInside`

Its outer source and target functions equal those of the original boxed value; instantiate the two deep-locality theorems at the selected site. Its sole outer edge label is the box with body value$[e :=$ otherValue$]$. For this label equation, unfold the site and replacement, simplify the equality test selecting the sole outer edge, and use reflexivity for the resulting body. The outer box is retained rather than replaced by the value operation itself.

Lean: `Linear.Examples.deep_replacement_source_unchanged`

Lean: `Linear.Examples.deep_replacement_target_unchanged`

Lean: `Linear.Examples.deep_replacement_body`

17.3 An input-rooted operation path

Give the single unit operation one term argument and no binders. Its corolla argument has type $* \to *$. The input, its only edge, and the output form a forward operation path: read this as the one-step reverse trace, check membership in the two singleton incidence lists, and end with an empty trace.

Lean: `Linear.Examples.argumentOperations`

Lean: `Linear.Examples.argument`

Lean: `Linear.Examples.argument_path`

Any reverse operation trace from its output back to that output has empty edge list. Apply the no-cycle theorem to the constructed input-rooted path, deriving the required unary condition from the typed signature. This exercises the connection between concrete graph typing and generic machine terminal uniqueness.

Lean: `Linear.Examples.no_cycle_after_argument`

17.4 Filling a hole inside a certified context

Choose the typed hole label with name zero, one term input, and no outputs. Its corolla singleHole is a depth-zero hypernet. It has unique holes because any two edge occurrences are equal: both are elements of Fin(1), so their underlying indices are zero. Pairing this graph with the proof constructs a context.

Lean: `Linear.Examples.holeLabel`

Lean: `Linear.Examples.singleHole`

Lean: `Linear.Examples.singleHole_unique`

Lean: `Linear.Examples.singleContext`

Box the graph with zero bound and free variables to obtain boxedHole, and box the certified context to obtain boxedContext. The unique-hole certificate in the latter is supplied by the general boxing theorem. Looking up the deep address consisting of the only outer edge and the only body edge returns the chosen hole label by definition.

Lean: `Linear.Examples.boxedHole`

Lean: `Linear.Examples.boxedContext`

Lean: `Linear.Examples.hole_is_deep`

The corresponding site descends into that box and chooses its shallow hole edge. Filling this site with the value corolla produces filledBox. Its outer label is the same zero-bound, zero-free box, now containing singleHole$[e := \mathsf{value}]$; unfold deep replacement and simplify the selected-edge equality test to prove this equation.

Lean: `Linear.Examples.boxedHoleSite`

Lean: `Linear.Examples.filledBox`

Lean: `Linear.Examples.filled_box_body`

Finally every edge of this filled body has no primitive hole label. Split on its edge tag. An old surviving edge would be an element of the singleton old carrier different from its only element, contradicting the surviving-edge certificate. An inserted value edge is an operation, whose primitive hole lookup is definitionally empty. Thus this example proves removal of the old hole, not merely the well-typedness of the filled graph.

Lean: `Linear.Examples.filled_body_no_holes`

18 Proof checking and remaining obligations

18.1 The two axiom audits

The file `Audit.lean` contains six `#print axioms` commands. They inspect counting-simulation soundness, the forward and converse simulation constructions, the two contextual sufficiency theorems, and the zero-step converse regression theorem. These diagnostic commands introduce no new definitions or proofs.

Lean: `Audit: abstract theorem dependencies`

The graph audit inspects shallow plugging, host and inserted path equivalence, plugging congruence, exact trace uniqueness, typed operation-path uniqueness, finite deep enumeration, unique holes under boxing, deep plugging and its other-label locality, and the self-loop and deep-replacement regression proofs.

Lean: `Graph.Audit: concrete theorem dependencies`

In the pinned snapshot, counting-simulation soundness, finite deep enumeration, and unique holes under boxing report only `propext` and `Quot.sound`. The other audited declarations additionally report `Classical.choice`. These are standard Lean logical axioms: propositional extensionality, quotient soundness, and classical choice, respectively. Neither audit reports `sorryAx` or a project-specific axiom.

Absence of project-specific axioms does not discharge a theorem's hypotheses. Specimen laws and initial lifting remain parameters of the abstract sufficiency results. They do not appear as named global axioms in the report. In contrast, the concrete graph constructions explicitly prove the invariant fields needed to construct their results. Neither fact supplies the missing bridge to a concrete UAM.

18.2 Reproducing the Lean and document checks

The snapshot pins Lean `4.35.0` in `lean-toolchain` and uses only the bundled `Std`. With that toolchain available, run

```
lake build
lake env lean Hypernets/Audit.lean
lake env lean Hypernets/Graph/Audit.lean
```

The script `scripts/check.sh` runs the build and both audits. Run it using `bash` with the script path as its argument. It selects the workspace-local macOS ARM compiler if present, and otherwise uses `lake` from the environment. The audits are checked separately from the root import.

The commentary's source checker recursively scans all Lean modules under `Hypernets/`, including nested graph modules, attributed declarations, noncomputable definitions, abbreviations, and anonymous examples. It verifies that each declaration has a source-fragment link, all linked ranges are valid, and each working source file equals the version at the pinned commit. Diagnostic audit ranges are linked separately. Run it with

```
python3 docs/mathematical-commentary/check_sources.py
```

This syntactic coverage check complements the mathematical exposition; it does not itself check the correctness of English explanations. Source links are immutable GitHub commit links, so future edits cannot move the referenced proof.

From the commentary project directory, `make` checks the links, compiles with `latexmk`, and exports the PDF to the repository's `output/pdf/` directory. The committed PDF can therefore be read without a TeX installation, and its editable sources and reproduction instructions are included alongside it.

18.3 What remains for the paper's concrete theorem

The following obligations remain before Theorem 6 can be claimed with its original hypotheses:

1. Prove a representation theorem relating the typed depth-indexed hierarchy with explicit binder metadata to the paper's set-based graph-labelled hypernets. Extend isomorphism recursively to identify independently renamed box bodies.
2. Complete the context algebra: associativity of plugging and preservation of global hole-label uniqueness under suitable freshness conditions. Shallow well-formedness, path locality, and addressed deep replacement are already proved.
3. Define concrete focussed UAM states, search and rewrite transitions, rootedness, determinism, and refocusing. Prove their structural transition laws. Concrete focus labels and operation traversal alone do not define that machine.
4. Define concrete pre-templates, specimens, quasi-specimens, output-closure, input-safety, and robustness, with the paper's typing and context restrictions.
5. Construct the abstract specimen interface from those objects, proving its rootedness, final-agreement, and context-only traversal laws, including the needed consequences of Lemma 39. Prove the safety and robustness bridges and initial lifting for concrete observations.
6. Instantiate the two checked contextual sufficiency results. Only this last instantiation with all concrete obligations discharged yields the target robustness theorem.

The present milestone proves both the abstract quantitative argument and a substantial concrete graph foundation. It does not yet establish that all hypotheses of the former hold for the universal abstract machine.

References

[1] Dan R. Ghica, Koko Muroya, and Todd Waugh Ambridge. *A robust graph-based approach to observational equivalence*. Logical Methods in Computer Science 21(2:8), 2025. Reference version: arXiv:1907.01257v5.

[2] *Hypernets in Lean 4*. Repository at revision `ce31471`.